

(Im)Possibility of Symmetric Encryption against Coordinated Algorithm Substitution Attacks and Key Exfiltration

Simone Colombo¹  and Damian Vizár² 

¹ King's College London, London, UK

`simone.colombo@kcl.ac.uk`

² CSEM, Neuchâtel, Switzerland

`damian.vizAr@csem.ch`

August 5, 2025

Abstract. A growing body of work addresses the security of cryptographic systems in the presence of mass surveillance, a threat made concrete by Snowden's revelations and the widespread use of spyware against journalists and activists. In this paper, we investigate the security of symmetric encryption faced with simultaneous algorithm substitution attacks (ASAs) and key exfiltration (KE). The security of symmetric encryption in presence of ASAs or KE alone was established but no result deals with their coordinated deployment. Yet, that is a necessary step to be made if we are to achieve actual security against mass surveillance. We formalize this setting, and prove that no scheme alone stands chance against coordinated ASA and KE, by describing a realistic attack. We then describe a new kind of schemes, which make use of externally supplied randomness. We formalize their security and give a construction which provably resists simultaneous ASAs and KE when paired with a verifiable source of randomness, with security bounds in the concrete security spirit.

Keywords: Provable Security, Mass Surveillance, Symmetric Encryption, Key Exfiltration, Algorithm Substitution

1 Introduction

In this paper, we study the security of symmetric encryption in the presence of *simultaneous* algorithm substitution attacks (ASAs) and key exfiltration (KE) via advanced persistent threats (APTs). Both APTs and ASAs enable adversaries to circumvent encryption, facilitating mass surveillance by resourceful entities, to which we refer as *Big Brother*. The idea of covertly subverting cryptographic

Work partially done while Colombo and Vizár where at EPFL. An abridged version of this paper appears in the proceedings of LATINCRYPT 2025. This is the full version.

systems was first introduced by Young and Yung under the term *kleptography* nearly three decades ago [53, 54]. The threat of mass surveillance enabled by sophisticated technical means has been made starkly evident by the Snowden revelations and the widespread deployment of spyware targeting journalists and activists [7, 28, 29, 51]. This concern has also been formally acknowledged by the IACR in its “Copenhagen Resolution” [32]. These developments have prompted a resurgence of interest in kleptographic research [15, 19, 27, 34, 42, 45–48].

ALGORITHM SUBSTITUTION ATTACKS. In an ASA, the executable code implementing, e.g., an encryption algorithm $\mathcal{E}$ gets replaced by the code of a subversion $\tilde{\mathcal{E}}$. The goal of $\tilde{\mathcal{E}}$ is to break confidentiality of communication, and remain undetected by the user while doing so. The target could be a closed-source cryptographic library [20] or a dedicated crypto-hardware [50], or possibly modifications of open source software at compile time [14].

Bellare, Paterson and Rogaway (BPR) formalized the goals of both Big Brother (to avoid detection by the users) and the users (to prevent the existence of useful subversions) in the context of ASAs against symmetric encryption [14]. They demonstrated an efficient and provably undetectable subversion for any symmetric scheme with randomized encryption algorithm. In the opposite direction, BPR showed that a large class of deterministic and stateful symmetric schemes is provably resistant to ASAs (assuming the subverted algorithm meets a very basic condition of concealment). The results of BPR indicate that *no randomized encryption scheme can resist ASAs*.

KEY EXFILTRATION THROUGH APTs. An APT, e.g., a malware like NSO’s Pegasus [51], infests a system for a long period and attempts to exfiltrate a user’s secret key [13] to Big Brother, negating the encryption. The Snowden revelations indicate, that the NSA is capable of planting APTs across a wide range of platforms [7]. An APT will however be able to communicate a limited amount of data (say 10 GB) to the outside world without being discovered [3]. This motivates the bounded retrieval model (BRM): with a very big secret key (e.g., 1 TB) the damage done by a limited leakage can be mitigated.

Bellare, Kane and Rogaway (BKR) formalized the security of symmetric encryption with leakage in the BRM [13]. They designed a big-key symmetric scheme that is both practically useful, and provably secure against KE in the random-oracle model (ROM)³. A crucial aspect of their scheme’s security is that *the encryption algorithm uses randomness* to render prior leakage useless.

THE PROBLEM. We address an open question posed by BKR [13]:

“Whether any defense against ASAs is possible in the big-key setting remains open.”

Indeed, it is implausible that Big Brother would refrain from coordinating its tools in a targeted attack. Yet, the two existing results that address ASAs and

³ BKR also present a less-efficient variant that achieves standard model security.

KE in isolation seem at odds: resisting ASAs requires determinism, whereas defeating KE necessitates randomness. We set out to answer the open question and find means of thwarting simultaneous ASAs and KE with minimal trust.

OUR CONTRIBUTION. We define SURV-LIND, a notion that captures security of symmetric encryption in face of simultaneous ASAs and KE in the ROM. We then describe the *derandomization* attack, confirming that no scheme, randomized or not, can be SURV-LIND secure. We further give a practical variant of the derandomization attack that can be used e.g. for identification of a subset of users of an online service at the time of login. Through the attack, we identify the Achilles heel of the setup: Big Brother’s freedom to manipulate randomness.

To sidestep this impossibility result, we propose a new big-key symmetric encryption scheme that uses external randomness. We formalize security with the RESIST notion in the ROM, which models simultaneous ASAs and key exfiltration, assuming that the randomness used by the encryption algorithm is generated by a subroutine free from attacker influence. We then design a big-key symmetric encryption scheme SES that is provably RESIST-secure in this setting. Our proof provides a concrete security bound [9], yielding a meaningful numerical estimate of actual security in practice.

Seemingly a definitional weakening, the explicit, incorruptible randomness in RESIST serves as model for an incorruptible random beacon, a concept introduced through the *unicorn* protocol by Lenstra and Wesolowski (LW) [36]. The *unicorn* protocol allows a variable number of parties to generate a provably uncontestable random value, even if we trust no other party but ourselves. When using *unicorn* to supply our construction SES with randomness, a user can defeat Big Brother who deploys simultaneous ASAs and APTs.

We finally provide a variant of the notion RESIST that incorporates the security game of *unicorn*, allowing the attacker to attempt to thwart incorruptible randomness generation. We prove the security of SES under this advanced notion, capturing all relevant attack vectors in a single advantage bound. For the sake of efficiency, we propose a variant of big-key symmetric encryption with external randomness, designed in an API-driven syntax with session support: while using external randomness for every encryption query yields a secure scheme, it is exceedingly inefficient. Our construction amortizes this overhead by reusing the costly randomness within a session. SES can thus be easily instantiated and, when paired with a suitable instance of the *unicorn* protocol, provides an efficient means of securing communication against mass surveillance. To our knowledge, SES paired with *unicorn* is the first scheme that provably resists simultaneous ASAs and KE in the ROM.

RELATED WORK. The pioneering work on kleptography was done by Young and Yung [53, 54]. BPR gave the first formal treatment of ASAs in symmetric encryption [14]. Bellare, Jaeger, and Kane improved BPR’s undetectable ASAs against randomized encryption by making them stateless (even less detectable) and more general [12]. Degabriele, Farshim, and Poettering argued that BPR’s assumption about subversions is too strong and relaxed it [23]. Dodis, Mironov,

and Stephen-Davidowitz proposed *cryptographic reverse firewalls*, which thwart ASAs by re-randomizing outgoing ciphertexts [24, 38]. These could also address the simultaneous ASAs and KE, however they require trusted networking devices. We did not adopt this approach, as our goal is a solution with minimal trust. Austrin et al. show that no security is possible if the random tape of an algorithm is tamperable [6]. Russell et al. investigate security when all algorithms can be subverted [46]. Several works study ASAs on various cryptographic primitives [5, 11, 15, 17, 47, 49]. The BRM is the result of a sequence of works [1, 2, 18, 22, 25, 26, 39]. BKR presented the first efficient symmetric encryption scheme that is computationally secure in the BRM [13]. We rely on encryption tidiness [40] to argue uniqueness of ciphertexts in our proofs. This is reminiscent of the attack where adversary tries to craft ciphertext tuples that correctly decrypt under two different keys [30], which is not thwarted by tidiness. That scenario is not relevant in our notions, where the adversary tries to exfiltrate a key without controlling it—here tidiness suffices as it implies per-key ciphertext uniqueness for any given key, which remains fixed in our security games. Hoang et al. also use an API-driven interface for symmetric encryption, though for a different purpose [31].

ORGANIZATION OF THE PAPER. In Section 2, we introduce notations and briefly review existing security notions. In Section 3, we present the SURV-LIND security notion, describe the derandomization attack, and illustrate its practical impact. In Section 4, we introduce the syntax of big-key symmetric encryption with sessions and define RESIST security. In Section 5, we describe a scheme paired with an honest source of randomness and prove its RESIST security. In Section 6, we present the *unicorn* protocol, which we prove can securely instantiate the honest source of randomness. Section A presents deferred definitions and lemmas from related work, Section B contains deferred proofs, and Section C presents algorithms and proofs for *unicorn*.

2 Preliminaries and Previous Security Definitions

NOTATIONS. We denote the set of all strings of n bits with $\{0, 1\}^n$ and with $\{0, 1\}^*$ the set of all finite strings. The length of a string s is denoted by $|s|$. The special symbol $\perp$ represents an undefined variable or an error. The empty string is represented by ε . The concatenation of two strings s, s' is denoted by $s||s'$. By $s[i]$ we denote i^{th} bit of a string s . We represent with $\langle s \rangle$ the efficient and injective encoding of a string s . For a set $\mathcal{V}$, we denote a vector as $\vec{v} \in \mathcal{V}^q$, with $\vec{v}[i] \in \mathcal{V}$ its i -th coordinate and with $|\vec{v}| = q$ its length. For a string s and a vector $\vec{P} \in (\mathbb{Z}_{|s|})^q$ we let $s[\vec{P}]$ denote $s[\vec{P}[1]]||\dots||s[\vec{P}[q]]$. We let $\mathbb{N}$ denote the set $\{0, 1, 2, \dots\}$ and $\mathbb{N}_{>0}$ denote $\mathbb{N} \setminus \{0\}$. We let $\mathbb{Z}_n$ denote both the set $\{0, 1, \dots, n-1\}$ and the corresponding ring with n elements. We further let $\mathbb{Z}_n^*$ denote both the multiplicative subgroup of $\mathbb{Z}_n$ and the corresponding set. For a prime p , we let $\text{QR}_p \subset \mathbb{Z}_p^*$ denote the set of quadratic residues modulo p . We let $x \leftarrow_s S$

denote uniform sampling of an element x from a set S (finite or endowed with a uniform distribution). We let $y \leftarrow A(x_1, \dots; r)$ denote running a randomized algorithm A on inputs $x_1, \dots$ and coins r to obtain y in a deterministic way, while $y \leftarrow \$ A(x_1, \dots)$ denotes running A with implicit coins. A random oracle $\text{RO}(s, l')$ maps every distinct input $s \in \{0, 1\}^*$ to an infinite string of uniformly and independently sampled bits, which is truncated to l' bits. For a set $\mathcal{S}$, we let $\text{Perm}(\mathcal{S})$ denote the set of all permutations over $\mathcal{S}$.

SYMMETRIC ENCRYPTION. We use the same syntax and correctness definitions as BPR [14]. A scheme for symmetric encryption is a triple $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$. The key space $\mathcal{K}$ is a finite nonempty set. The possibly randomized and stateful encryption algorithm $\mathcal{E} : \mathcal{K} \times \mathcal{M} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathcal{C} \times \mathcal{S}$, maps a key K , a message M , associated data A and a state σ from their respective domains, which are all subsets of $\{0, 1\}^*$, to a ciphertext and an updated state $(C, \sigma') \leftarrow \$ \mathcal{E}(K, M, A, \sigma)$. The decryption algorithm $\mathcal{D} : \mathcal{K} \times \mathcal{C} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathcal{M} \cup \{\perp\} \times \mathcal{S}$ is a deterministic algorithm mapping a tuple (K, C, A, τ) to a recovered plaintext and an updated state $(M, \tau') \leftarrow \mathcal{D}(K, C, A, \tau)$.

We require that $\forall M \in \mathcal{M} : M \in \{0, 1\}^m \Rightarrow \{0, 1\}^m \subseteq \mathcal{M}$. We further require that for all $K \in \mathcal{K}$, $M \in \mathcal{M}$, $A \in \mathcal{A}$ and $\sigma \in \mathcal{S}$ we have $|\mathcal{E}(K, M, A, \sigma)| = |M| + \mu$ where μ is a constant called *stretch* of Π . We say that Π is stateless if $\mathcal{S} = \{\varepsilon\}$. In this case, we may drop the state input and output from the notation.

CORRECTNESS. A scheme for symmetric encryption is *correct*, if the receiver that decrypts the incoming ciphertexts in the correct order always obtains the same messages as the sender encrypted, i.e., if for all $q > 1$ and all $K \in \mathcal{K}$, $M_1, \dots, M_q \in \{0, 1\}^*$ and $A_1, \dots, A_q \in \{0, 1\}^*$ the following holds:

$$\Pr \left[(\forall i, C_i \neq \perp) \wedge (\exists i, M_i \neq M'_i) \left| \begin{array}{l} \sigma_0, \tau_0 \leftarrow \varepsilon; \\ \textbf{for } i = 1, \dots, q \textbf{ do} \\ \quad (C_i, \sigma_i) \leftarrow \$ \mathcal{E}(K, M_i, A_i, \sigma_{i-1}) \\ \quad (M'_i, \tau_i) \leftarrow \mathcal{D}(C_i, A_i, \tau_{i-1}) \end{array} \right. \right] = 0$$

SECURITY OF SYMMETRIC ENCRYPTION. In this work we use two security definitions for symmetric encryption: left or right (LoR), and real or random (RoR) indistinguishability (IND) under chosen plaintext attack (CPA).

Definition 1 (IND-CPA and IND\$-CPA). Let $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ be an encryption scheme. We define the advantage of an adversary $\mathcal{A}$ at winning the IND-CPA (resp. IND\$-CPA) game (Fig. 1) as $\text{Adv}_{\Pi}^{\text{ind}}(\mathcal{A}) = 2 \Pr[\text{IND-CPA}_{\Pi}^{\mathcal{A}} \Rightarrow 1] - 1$ (resp. as $\text{Adv}_{\Pi}^{\text{ind\$}}(\mathcal{A}) = 2 \Pr[\text{IND\$-CPA}_{\Pi}^{\mathcal{A}} \Rightarrow 1] - 1$).

An encryption scheme Π is said to be IND-CPA-secure (resp. IND\$-CPA-secure) when this advantage is small for all adversaries $\mathcal{A}$ with reasonable resources. It is well-known that $\text{IND\$-CPA} \Rightarrow \text{IND-CPA}$ [8, 10, 44] and we will use this result in the rest of the paper without proof.

Game	$\text{IND-CPA}_{\Pi}^{\mathcal{A}}$	$\text{IND\$-CPA}_{\Pi}^{\mathcal{A}}$	Oracle	$\text{ENC}(M_0, M_1, A)$	Oracle	$\text{ENC}(M, A)$
$b \leftarrow_{\$} \{0, 1\}; \sigma \leftarrow \varepsilon$			$(C, \sigma) \leftarrow_{\$} \mathcal{E}(K, M_b, A, \sigma)$		$(C_0, \sigma) \leftarrow_{\$} \mathcal{E}(K, M, A, \sigma)$	
$K \leftarrow_{\$} \mathcal{K}$			return C		$C_1 \leftarrow_{\$} \{0, 1\}^{ C_0 }$	
$b' \leftarrow \mathcal{A}^{\text{ENC,RO}}(L, \sigma)$					return C_b	
return $(b = b')$						

Fig. 1: Games used to define security of symmetric encryption. The IND-CPA game uses the $\text{ENC}(M_0, M_1, A)$ oracle, whereas $\text{IND\$-CPA}_{\Pi}^{\mathcal{A}}$ uses the $\text{ENC}(M, A)$ oracle.

2.1 Algorithm Substitution Attacks

BPR formalize an ASA subverting the encryption algorithm of a symmetric scheme. They assume the subversion to be equipped with a master key, that is used to subliminally communicate with Big Brother, and allow it to decrypt the plaintexts. We summarize the model that BPR use.

SUBVERSIONS. A subversion of a scheme $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ is a triple $\tilde{\Pi} = (\tilde{\mathcal{K}}, \tilde{\mathcal{E}}, \tilde{\mathcal{D}})$. The master-key space $\tilde{\mathcal{K}}$ is a finite non-empty set. The subverted encryption algorithm $\tilde{\mathcal{E}} : \tilde{\mathcal{K}} \times \mathcal{K} \times \mathcal{M} \times \mathcal{A} \times \tilde{\mathcal{S}} \times \mathcal{I} \rightarrow \mathcal{C} \times \tilde{\mathcal{S}}$, which is possibly randomized and stateful, maps a tuple $(\tilde{K}, K, M, A, \sigma, i)$ to a ciphertext and an updated state (C, σ') , where $i \in \mathcal{I}$ is a public value uniquely identifying a user (e.g., a MAC address). All the domains of input and output arguments are subsets of $\{0, 1\}^*$. The plaintext-recovery algorithm $\tilde{\mathcal{D}} : \tilde{\mathcal{K}} \times \mathcal{C}^* \times \mathcal{A}^* \times \mathcal{I} \rightarrow \mathcal{M}^* \cup \perp$ takes a master key $\tilde{K}$, a vector of ciphertexts $\vec{C}$ and of associated data $\vec{A}$ each, and a user identifier i to a vector messages $\vec{M}$ or to $\perp$ if the plaintext recovery fails. We do not further discuss how $\tilde{\mathcal{D}}$ can be constructed, but rather study how any such $\tilde{\mathcal{D}}$ can be thwarted.

DECRYPTABILITY. BPR assume any subversion must meet the *decryptability condition* [14], as a basic requirement for concealment; failure do so is likely to lead to detection. A subversion $\tilde{\Pi} = (\tilde{\mathcal{K}}, \tilde{\mathcal{E}}, \tilde{\mathcal{D}})$ satisfies the decryptability condition relative to $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ if $\Pi' = (\tilde{\mathcal{K}} \times \mathcal{K}, \tilde{\mathcal{E}}, \mathcal{D}')$ is a correct encryption scheme where $\mathcal{D}'$ is defined by $\mathcal{D}'((\tilde{K}, K), C, A, \sigma) = \mathcal{D}(K, C, A, \sigma)$.

Less stringent variants of the decryptability condition have been proposed (e.g., [4, 23]), extending the set of “valid” subversions. We adhere to BPR’s definition of decryptability, as we do not focus on the nuances of different decryptability notions, and because BPR’s measures defeat subversions under all known variants [23].

DEFEATING ASAS. BPR ask that Big Brother, in possession of the master-key, should not only be unable to decrypt the communication, but rather unable to tell apart the real scheme from the subverted one in a chosen plaintext attack.

Definition 2 (SURV [14]). Let $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ be an encryption scheme and $\tilde{\Pi} = (\tilde{\mathcal{K}}, \tilde{\mathcal{E}}, \tilde{\mathcal{D}})$ its subversion. We define the advantage of Big Brother $\mathcal{B}$ at winning the SURV game of Fig. 2 as $\text{Adv}_{\Pi, \tilde{\Pi}}^{\text{SURV}}(\mathcal{B}) = 2 \Pr[\text{SURV}_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow 1] - 1$.

An encryption scheme Π is said to be secure against surveillance when this advantage is small for all subversions $\tilde{\Pi}$ of Π that meet the decryptability condition and all adversaries $\mathcal{B}$ with reasonable resources.

Game $\text{SURV}_{\Pi, \tilde{\Pi}}^{\mathcal{B}}$	Oracle $\text{KEY}(i)$	Oracle $\text{ENC}(M, A, i)$
$b \leftarrow_{\$} \{0, 1\}$	if $(K_i = \perp)$ then	if $(K_i = \perp)$ then return $\perp$
$\tilde{K} \leftarrow_{\$} \tilde{\mathcal{K}}$	$K_i \leftarrow_{\$} \mathcal{K}$	if $(b = 1)$ then $(C, \sigma_i) \leftarrow_{\$} \mathcal{E}(K_i, M, A, \sigma_i)$
$b' \leftarrow \mathcal{B}^{\text{KEY}, \text{ENC}}(\tilde{K})$	$\sigma_i \leftarrow \varepsilon$	else $(C, \sigma_i) \leftarrow_{\$} \tilde{\mathcal{E}}(\tilde{K}, K_i, M, A, \sigma_i, i)$
return $(b = b')$	return ε	return C

Fig. 2: Game used to define surveillance security of subversion [14].

BPR showed that any scheme that has *unique ciphertexts* (Definition 13 in Section A) will provably resist to ASAs: if only a single ciphertext decrypts to any message (with other inputs to $\mathcal{E}$ fixed), a subversion $\tilde{\mathcal{E}}$ can either implement $\mathcal{E}$ or break the decryptability condition. We remark that due to the correctness condition, any unique-ciphertext scheme is deterministic.

2.2 Key Exfiltration

BKR model an attack by an APT in the BRM. BKR consider symmetric schemes with a very big key. They formalize the security of such schemes exposed to an APTs in the BRM and design a provably secure, practical construction.

BIG-KEY SYMMETRIC ENCRYPTION. BKR’s syntax of big-key symmetric encryption (BKSE) is a special case of the syntax defined for symmetric encryption: BKR consider stateless schemes with no associated data. A scheme for big-key symmetric encryption (BKSE) is a triple $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$, the key space being $\mathcal{K} = \{0, 1\}^k$ with k “big”, say 1 TB. We use bold symbols to emphasize the big-key setting. The correctness condition is required to be fulfilled. For the sake of efficiency and usability, BKR also require *locality* of the encryption, i.e. the encryption algorithm should only use a “small” fraction of the big key $\mathbf{K}$ upon every invocation.

SECURITY MODEL FOR KEY EXFILTRATION ATTACKS. BKR model the security of BKSE in presence of an APT as a LoR IND-CPA game in the ROM, in which Big Brother $\mathcal{B}$ chooses a leakage function Lk that takes a big key $\mathbf{K}$, has access to the random oracle⁴ and outputs a leakage L with $|L| = l < k$. The amount of

⁴ Letting Lk access the RO is necessary to eliminate pathological constructions [13].

leakage l is a parameter of $\mathcal{B}$. The adversary learns the leakage at the *beginning* of the game which then proceeds as a conventional LoR experiment.

Game $\text{LIND}_{\Pi}^{\mathcal{B}}$	Oracle $\text{ENC}(M_0, M_1)$
$(\text{Lk}, \sigma) \leftarrow \mathcal{B}^{\text{RO}}$	$C \leftarrow_{\$} \mathcal{E}^{\text{RO}}(\mathbf{K}, M_b)$
$\mathbf{K} \leftarrow_{\$} \{0, 1\}^k$	return C
$L \leftarrow \text{Lk}^{\text{RO}}(\mathbf{K})$	
$b \leftarrow_{\$} \{0, 1\}$	
$b' \leftarrow \mathcal{B}^{\text{ENC,RO}}(L, \sigma)$	
return $(b = b')$	

Fig. 3: Game used to define security of a big-key encryption scheme $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ in presence of key exfiltration. The adversarial leakage function takes $\mathbf{K}$ as input, has access to the RO and outputs a leakage L with $|L| = l$ [13].

Definition 3 ([13]). *Let $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ be a big-key encryption scheme. We define the advantage of Big Brother $\mathcal{B}$ the LIND game defined in Fig. 3 as $\text{Adv}_{\Pi}^{\text{lind}}(\mathcal{B}) = 2 \Pr[\text{LIND}_{\Pi}^{\mathcal{B}} \Rightarrow 1] - 1$.*

A big-key encryption scheme Π is said to be secure in presence of key exfiltration when this advantage is small for all adversaries $\mathcal{B}$ with reasonable resources.

KEY ENCAPSULATION. BKR reduce the security of BKSE to the security of big-key encapsulation. A big-key encapsulation algorithm $\mathcal{S} : \mathcal{K} \times \{0, 1\}^r \rightarrow \{0, 1\}^{\kappa}$ takes a big key $\mathbf{K}$ and a selector/seed R to a derived key K with $\kappa < k$. We let $\mathcal{S}$ access the random oracle. A big-key encapsulation scheme is secure, if the derived keys are indistinguishable from uniform strings in presence of leakage.

Definition 4 ([13]). *Let $\mathcal{S} : \mathcal{K} \times \{0, 1\}^r \rightarrow \{0, 1\}^{\kappa}$ be a big-key encapsulation scheme. The advantage of an adversary $\mathcal{B}$ at winning the game KEY (Fig. 4) is defined as $\text{Adv}_{\mathcal{S}}^{\text{key}}(\mathcal{B}) = 2 \Pr[\text{KEY}_{\mathcal{S}}^{\mathcal{B}} \Rightarrow 1] - 1$.*

A big-key encapsulation algorithm $\mathcal{S}$ is said to be secure in presence of key-exfiltration when this advantage is small for all adversaries $\mathcal{B}$ with reasonable resources. BKR designed a provably secure key encapsulation scheme XKEY (right of Fig. 4), whose security follows from Lemma 6 in Section A.

RESISTING THE KE. BKR proved that combining a secure big-key encapsulation algorithm $\mathcal{S}$ with a secure symmetric encryption Π in a natural way yields a secure BKSE $\text{SE}[\Pi, \mathcal{S}]$, for which $\text{Adv}_{\text{SE}}^{\text{lind}}(\mathcal{B}) \leq \text{Adv}_{\mathcal{S}}^{\text{key}}(\mathcal{A}_2) + q \cdot \text{Adv}_{\Pi}^{\text{lind}}(\mathcal{A}_1)$ for any adversary $\mathcal{B}$ that makes at most q queries to its ENC oracle and leaks at most l bits, such that $\mathcal{A}_1$ makes only a single ENC query and has about the same running time as $\mathcal{B}$. $\mathcal{A}_2$ makes at most q queries DERIVE queries, runs in about

Game $\text{KEY}_{\mathcal{S}}^{\mathcal{B}}$	Oracle $\text{DERIVE}()$	Algorithm $\text{XKEY}_{k,\kappa,P,r}^{\text{RO}}(\mathbf{K}, R)$
$(\text{Lk}, \sigma) \leftarrow \mathcal{B}^{\text{RO}}$	$R \leftarrow \mathcal{S} \{0, 1\}^r$	for $i \leftarrow 1, \dots, P$ do
$\mathbf{K} \leftarrow \mathcal{S} \{0, 1\}^k$	if $(b = 0)$ then	$\mathbf{P}[i] \leftarrow \text{RO}(\langle R, i, 0 \rangle, \log_2(k))$
$L \leftarrow \text{Lk}^{\text{RO}}(\mathbf{K})$	$K \leftarrow \mathcal{S} \{0, 1\}^\kappa$	$J \leftarrow \mathbf{K}[\mathbf{P}]$
$b \leftarrow \mathcal{S} \{0, 1\}$	else	$K \leftarrow \text{RO}(\langle R, J, 1 \rangle, \kappa)$
$b' \leftarrow \mathcal{B}^{\text{DERIVE}, \text{RO}}(L, \sigma)$	$K \leftarrow \mathcal{S}^{\text{RO}}(\mathbf{K}, R)$	return K
return $(b = b')$	return (R, K)	

Fig. 4: **(Left)** Game used to define security of a big-key encapsulation algorithm $\mathcal{S}$ in presence of key exfiltration [13]. **(Right)** Provably secure key encapsulation algorithm XKEY . The parameters k, κ, P, r are the length of the big key, the length of the derived key, the number of probed bits of the big key and the size of the selector respectively, and $\langle \cdot \rangle$ is an efficient, injective encoding [13].

the same time as $\mathcal{B}$ and makes the same number of RO queries both before and after receiving the leakage as $\mathcal{B}$.

3 Simultaneous ASAs and KE

A natural goal for any real-world application of symmetric encryption would be to resist both ASAs and KE simultaneously. Yet, the counter-measures against isolated ASAs proposed by BPR and those against isolated KE proposed by BKR seem to be contradicting: it appears that a complex defense against Big Brother is not possible. In this section, we formally confirm this intuition. We define the notion SURV-LIND, a natural extension of BPR’s SURV notion that incorporates key exfiltration attacks. We then demonstrate in Section 3.4 a generic attack, which attacks *any* BKSE scheme, randomized or not, with high SURV-LIND advantage.

3.1 Equivalent definitions for surveillance and KE security

Before defining the notion SURV-LIND, we present alternative definitions for surveillance security of subversion (Definition 5) and key exfiltration security (Definition 6). The original definitions (Definitions 2 and 3) use a LoR approach, while in this section we present a RoR approach and show that our alternative security notions are equivalent (in the case of surveillance) or imply (in the case of KE security) the original ones. Looking ahead, we will build our SURV-LIND notion on this equivalent definitions.

Definition 5 (SURV\$). Let $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ be an encryption scheme and $\tilde{\Pi} = (\tilde{\mathcal{K}}, \tilde{\mathcal{E}}, \tilde{\mathcal{D}})$ its subversion. We define the advantage of Big Brother $\mathcal{B}$ at winning the game SURV\$ (defined in Fig. 5) as

$$\text{Adv}_{\Pi, \tilde{\Pi}}^{\text{srV\$}}(\mathcal{B}) = 2 \Pr[\text{SURV\$}_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow 1] - 1.$$

Game $\text{SURV\$}_{\Pi, \tilde{\Pi}}^{\mathcal{B}}$	Oracle $\text{KEY}(i)$	Oracle $\text{ENC}(M, A, i)$
$b \leftarrow \$ \{0, 1\}$	if $(K_i = \perp)$ then	if $(K_i = \perp)$ then return $\perp$
$\tilde{K} \leftarrow \$ \tilde{\mathcal{K}}$	$K_i \leftarrow \$ \mathcal{K}$	$(C_0, \sigma_i) \leftarrow \$ \tilde{\mathcal{E}}(\tilde{K}, K_i, M, A, \sigma_i, i)$
$b' \leftarrow \mathcal{B}^{\text{KEY}, \text{ENC}}(\tilde{K})$	$\sigma_i \leftarrow \varepsilon$	$C_1 \leftarrow \$ \{0, 1\}^{ C_0 }$
return $(b = b')$	return ε	return C_b

Fig. 5: ROR ciphertext game used to define surveillance security of subversion.

RELATION BETWEEN SURV NOTIONS. Proposition 1 shows that SURV\$ security is equivalent to SURV security.

Proposition 1 ($\text{SURV\$} \xleftrightarrow{\text{IND\$-CPA}} \text{SURV}$). *Let $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ be an IND-CPA\$-secure (Definition 1) encryption scheme and $\tilde{\Pi} = (\tilde{\mathcal{K}}, \tilde{\mathcal{E}}, \tilde{\mathcal{D}})$ its subversion. Then for any SURV adversary $\mathcal{B}$, there exists a SURV\$ adversary $\mathcal{B}'$ that makes the same number of KEY and ENC queries as $\mathcal{B}$, has the same data complexity as $\mathcal{B}$ and for which $|\text{Adv}_{\Pi, \tilde{\Pi}}^{\text{SRV}}(\mathcal{B}') - \text{Adv}_{\Pi, \tilde{\Pi}}^{\text{SRV\$}}(\mathcal{B})| \leq \text{Adv}_{\Pi}^{\text{ind\$-cpa}}(\mathcal{A})$, where $\mathcal{A}$ is an adversary against the IND\$-CPA game for Π that makes the same number of ENC as $\mathcal{B}$ and has the same data complexity as $\mathcal{B}$.*

Proof. Let $P(\text{G}, \bar{b}) = \Pr[\mathcal{B}^{\text{G}} \Rightarrow 1 | b = \bar{b}]$, i.e., the probability that adversary $\mathcal{B}$ outputs 1 after playing with game G given that b in game G is equal to $\bar{b}$.

By definition we have that $\text{Adv}_{\Pi, \tilde{\Pi}}^{\text{SRV}}(\mathcal{B}) = P(\text{SURV}, 1) - P(\text{SURV}, 0)$ and $\text{Adv}_{\Pi, \tilde{\Pi}}^{\text{SRV\$}}(\mathcal{B}) = P(\text{SURV\$}, 1) - P(\text{SURV\$}, 0)$. By construction we know that $P(\text{SURV}, 0) = P(\text{SURV\$}, 0)$, as when $b = 0$ the two games are identical. Therefore we have

$$|\text{Adv}_{\Pi, \tilde{\Pi}}^{\text{SRV}}(\mathcal{B}) - \text{Adv}_{\Pi, \tilde{\Pi}}^{\text{SRV\$}}(\mathcal{B})| = |P(\text{SURV}, 1) - P(\text{SURV\$}, 1)| \leq \text{Adv}_{\Pi}^{\text{ind\$-cpa}}(\mathcal{A}),$$

where the inequality follows from the observation that the difference between SURV and SURV\$ when $b = 1$ is identical to the IND\$-CPA game of Fig. 1. $\square$

Definition 6 (LIND\$). *Let $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ be a big-key encryption scheme. We define the advantage of Big Brother $\mathcal{B}$ at winning the game LIND\$ (defined in Fig. 6) as*

$$\text{Adv}_{\Pi}^{\text{lind\$}}(\mathcal{B}) = 2 \Pr[\text{LIND\$}_{\Pi}^{\mathcal{B}} \Rightarrow 1] - 1.$$

RELATION BETWEEN LIND NOTIONS. Proposition 2 shows that LIND\$ security implies LIND security.

Proposition 2 ($\text{LIND\$} \implies \text{LIND}$). *Let the triple $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ be a big-key encryption (BKSE) scheme (see Section 2.2). Then for any LIND adversary $\mathcal{B}$, there exists a LIND\$ adversary $\mathcal{B}'$ that makes the same number*

Game $\text{LIND\$}_{\Pi}^{\mathcal{B}}$	$\text{LIND-ROR}_{\Pi}^{\mathcal{B}}$	Oracle $\text{ENC}(M)$	Oracle $\overline{\text{ENC}}(M)$
$(\text{Lk}, \sigma) \leftarrow \mathcal{B}^{\text{RO}}$		$C_0 \leftarrow_{\$} \mathcal{E}^{\text{RO}}(\mathbf{K}, M)$	$C_0 \leftarrow_{\$} \mathcal{E}^{\text{RO}}(\mathbf{K}, M)$
$\mathbf{K} \leftarrow_{\$} \{0, 1\}^k$		$C_1 \leftarrow_{\$} \{0, 1\}^{ C_0 }$	$M' \leftarrow_{\$} \{0, 1\}^{ M }$
$L \leftarrow \text{Lk}^{\text{RO}}(\mathbf{K})$		return C_b	$C_1 \leftarrow_{\$} \mathcal{E}^{\text{RO}}(\mathbf{K}, M')$
$b \leftarrow_{\$} \{0, 1\}$			return C_b
$b' \leftarrow \mathcal{B}^{\text{ENC,RO}}(L, \sigma)$			
return $(b = b')$			

Fig. 6: Real-or-random ciphertext games used to define security of a big-key encryption scheme $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ in presence of key exfiltration. The game $\text{LIND-ROR}_{\Pi}^{\mathcal{B}}$ uses the $\overline{\text{ENC}}(M)$ oracle and it is used in the proof of Proposition 2.

of RO and ENC queries as $\mathcal{B}$, has the same data complexity as $\mathcal{B}$ and has $\text{Adv}_{\Pi}^{\text{lind}}(\mathcal{B}') \leq 4 \cdot \text{Adv}_{\Pi}^{\text{lind\$}}(\mathcal{B})$

Proof. The proofs first proves that $\text{Adv}_{\Pi}^{\text{lind-ror}}(\mathcal{B}) \leq 2 \cdot \text{Adv}_{\Pi}^{\text{lind\$}}(\mathcal{B}')$ and then relies on proof techniques from Bellare et al. [10, Theorem 1] to show that $\text{Adv}_{\Pi}^{\text{lind}}(\mathcal{B}) \leq 2 \cdot \text{Adv}_{\Pi}^{\text{lind-ror}}(\mathcal{B})$. We present the full proof in Section B.1. $\square$

3.2 Security Model

SYNTAX AND SUBVERSIONS. A BKSE scheme $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ is now a symmetric scheme defined in Section 2, but the keyspace $\mathcal{K} = \{0, 1\}^k$ is comprised of “big” keys. In particular, it can now be stateful and can process associated data. We require that Π meets the constant stretch and correctness properties. A subversion of a BKSE scheme $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ is a triple $\tilde{\Pi} = (\tilde{\mathcal{K}}, \tilde{\mathcal{E}}, \tilde{\mathcal{D}})$ defined in Section 2.1. The decryptability condition applies as defined in Section 2.1.

SECURITY. We capture the security of BKSE in presence of ASAs and key leakage in the notion SURV-LIND in the ROM. It is a multi-key RoR indistinguishability game that allows Big Brother $\mathcal{B}$ to both choose a leakage function to be applied to the users’ big keys and also substitute the encryption algorithm. As in BKR, the leakage from every key $\mathcal{B}$ wishes to attack is computed only once, before $\mathcal{B}$ can ask any encryption queries for this key. We leave the study of adaptive leakage to future work. The name SURV-LIND is a nod to the fact that the notion intends to be an extension of the SURV notion via the SURV-ROR notion that incorporates the LIND notion.

Definition 7 (SURV-LIND). Let $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ be a BKSE scheme and $\tilde{\Pi} = (\tilde{\mathcal{K}}, \tilde{\mathcal{E}}, \tilde{\mathcal{D}})$ its subversion. We define the advantage of Big Brother $\mathcal{B}$ at winning the game SURV-LIND (Fig. 7) as

$$\text{Adv}_{\Pi, \tilde{\Pi}}^{\text{srv-lind}}(\mathcal{B}) = 2 \Pr[\text{SURV-LIND}_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow 1] - 1.$$

A BKSE scheme Π is said to be secure against surveillance and key-leakage when this advantage is small for all subversions $\tilde{\Pi}$ of Π that meet the decryptability condition and all adversaries $\mathcal{B}$ with reasonable resources.

Game SURV-LIND $_{\Pi, \tilde{\Pi}}^{\mathcal{B}}$	Oracle ENC(M, A, i)	Oracle LEAK(i)
$\tilde{K} \leftarrow_{\$} \tilde{\mathcal{K}}$	if ($K_i = \perp$) then return $\perp$	if ($K_i = \perp$) then
$(Lk, \sigma) \leftarrow \mathcal{B}^{\text{RO}}(\tilde{K})$	$(C_0, \sigma_i) \leftarrow_{\$} \tilde{\mathcal{E}}^{\text{RO}}(\tilde{K}, K_i, M, A, \sigma_i, i)$	$K_i \leftarrow_{\$} \{0, 1\}^k$
$b \leftarrow_{\$} \{0, 1\}$	$C_1 \leftarrow_{\$} \{0, 1\}^{ C_0 }$	$\sigma_i \leftarrow \varepsilon$
$b' \leftarrow \mathcal{B}^{\text{LEAK, ENC, RO}}(\tilde{K}, \sigma)$	return C_b	$L \leftarrow_{\$} Lk^{\text{RO}}(K_i)$
return ($b = b'$)		return L
		return $\perp$

Fig. 7: Game used to define security of a big-key encryption scheme $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ in presence of key exfiltration and algorithm substitution by a subversion $\tilde{\Pi}$. The adversarial leakage function takes K_i as input for $i \in \mathcal{I}$, has access to the RO and outputs a leakage L with $|L| = l$.

3.3 Relations with Previous Notions

In this section we show that our SURV-LIND notions implies SURV\$ and LIND\$ security. This and the results of Section 3.1 imply that SURV-LIND security implies also SURV security and LIND security. The relations in the other directions are clarified by our impossibility result in Section 3.4.

RELATION WITH LIND AND LIND-\$ NOTIONS. Proposition 3 shows that SURV-LIND security implies LIND\$ security, formally establishing that SURV-LIND notion extends the LIND\$ notion. Proposition 2 shows that LIND\$ security implies LIND, therefore SURV-LIND $\implies$ LIND, i.e, SURV-LIND formally extends the BKR's LIND notion (Definition 3).

Proposition 3 (SURV-LIND $\implies$ LIND\$). *Let $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ be a BKSE scheme with $\mathcal{S} = \mathcal{A} = \emptyset$. Then for any LIND\$ adversary $\mathcal{B}$, there exists a SURV-LIND adversary $\mathcal{B}'$ that makes a single LEAK query, the same number of RO and ENC queries as $\mathcal{B}$, has the same data complexity as $\mathcal{B}$ and has $\text{Adv}_{\Pi, \tilde{\Pi}}^{\text{srv-lind}}(\mathcal{B}') = \text{Adv}_{\Pi}^{\text{lind-ror}}(\mathcal{B})$, where $\tilde{\Pi} = (\tilde{\mathcal{K}}, \tilde{\mathcal{E}}, \tilde{\mathcal{D}})$ is a dummy subversion with $\tilde{\mathcal{K}} = \emptyset$, and $\tilde{\mathcal{E}}(\tilde{K}, K, M, A, \sigma, i) = \mathcal{E}(K, M, A, \sigma)$.*

Proof. Adversary $\mathcal{B}'$ gets the leakage function from $\mathcal{B}$ and uses it as its own. It then picks any identifier i , asks LEAK(i) to obtain the leakage L and passes it to $\mathcal{B}$. Then, $\mathcal{B}'$ answers all ENC queries made by $\mathcal{B}$ with its own oracle, using the identifier i . When $\mathcal{B}$ outputs its final decision, $\mathcal{B}'$ simply forwards it. Because the subverted encryption algorithm $\tilde{\mathcal{E}}$ implements the actual big-key encryption $\mathcal{E}$, the simulation of LIND-ROR game for $\mathcal{B}$ is perfect. $\square$

RELATION WITH SURV NOTION. Proposition 3 shows that SURV-LIND $\Rightarrow$ SURV\$. Proposition 1 states that SURV\$ security implies SURV, therefore we have that SURV-LIND $\Rightarrow$ SURV, i.e., SURV-LIND formally extends BPR's SURV notion (Definition 2).

Proposition 4 (SURV-LIND $\Rightarrow$ SURV\$). *Let $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ be a BKSE scheme and $\tilde{\Pi} = (\tilde{\mathcal{K}}, \tilde{\mathcal{E}}, \tilde{\mathcal{D}})$ its subversion. Then for any SURV\$ adversary $\mathcal{B}$, there exists a SURV-LIND adversary $\mathcal{B}'$ that makes a single LEAK query to initialize a target key, the same number of RO and ENC queries as $\mathcal{B}$, has the same data complexity as $\mathcal{B}$ and has $\text{Adv}_{\Pi, \tilde{\Pi}}^{\text{srv-lind}}(\mathcal{B}') = \text{Adv}_{\Pi}^{\text{surv}}(\mathcal{B})$.*

Proof. Adversary $\mathcal{B}'$ sets a dummy leakage function. Whenever $\mathcal{B}$ queries the KEY oracle with i , $\mathcal{B}'$ forwards the query to its own LEAK oracle to initialize $\mathbf{K}_i$ and ignores the leakage L that the oracle returns. $\mathcal{B}'$ answers all ENC queries made by $\mathcal{B}$ with its own oracle. Because the ENC oracle of $\mathcal{B}'$ is identical to the ENC oracle of $\mathcal{B}$, the simulation of SURV\$ game for $\mathcal{B}$ is perfect. $\square$

3.4 Defeat by Big Brother

We give a generic attack that achieves a high SURV-LIND advantage for *any* BKSE scheme. We call this attack *derandomization* attack we present it in Fig. 8.

Algorithm $\mathcal{B}_{\text{drnd}}(\tilde{K}, \tau)$	Algorithm $\tilde{\mathcal{E}}^{\text{RO}}(\tilde{K}, \mathbf{K}_i, M, A, \sigma, i)$
if $(\tau = \perp)$ then	if $(\sigma = \varepsilon)$ then
$M \leftarrow_{\$} \{0, 1\}^\nu$	$\tilde{\sigma} \leftarrow 0; \bar{\sigma} \leftarrow \varepsilon$
return $(\text{Lk}_{i, \tilde{K}, M}, M)$	else
else	parse σ as $\tilde{\sigma}, \bar{\sigma}$
$M \leftarrow \tau$	$\tilde{\sigma} \leftarrow \tilde{\sigma} + 1$
$i \leftarrow_{\$} \mathcal{I}$	$r \leftarrow \text{RO}(\langle i, \tilde{K}, \tilde{\sigma} \rangle, r)$
$A \leftarrow \varepsilon$	$(C, \bar{\sigma}) \leftarrow \mathcal{E}^{\text{RO}}(\mathbf{K}_i, M, A, \sigma; r)$
$L \leftarrow \text{LEAK}(i)$	return $C, \langle \tilde{\sigma}, \bar{\sigma} \rangle$
$C \leftarrow \text{ENC}(M, A, i)$	Algorithm $\text{Lk}_{i, \tilde{K}, M}^{\text{RO}}(\mathbf{K}_i)$
$b' \leftarrow (L \neq \text{RO}(\langle C \rangle, l))$	$\sigma \leftarrow \varepsilon$
return b'	$r \leftarrow \text{RO}(\langle i, \tilde{K}, 0 \rangle, r)$
	$(C, \sigma') \leftarrow \mathcal{E}^{\text{RO}}(\mathbf{K}_i, M, \varepsilon, \sigma; r)$
	return $\text{RO}(\langle C \rangle, l)$

Fig. 8: The adversary, the leakage function and the subverted encryption algorithm that achieve high SURV-LIND advantage against any BKSE scheme. $\langle \cdot \rangle$ is an efficient, injective encoding and $\nu = \min\{|M| \text{ for } M \in \mathcal{M}\}$.

$\text{Lk}_{i,\tilde{K},\mathcal{V}}^{\text{RO}}(K)$
$\sigma \leftarrow \varepsilon$
$r \leftarrow \text{RO}(\langle i, \tilde{K}, 0 \rangle, r)$
$L \leftarrow \varepsilon$
for $i \leftarrow 1, \dots, u$ do
$(C_i, \sigma') \leftarrow \mathcal{E}^{\text{RO}}(K, V_i, \bar{A}, \sigma; r)$
$L \leftarrow L \ C_i$
return $\text{left}_l(L)$

Fig. 9: The leakage function $\text{Lk}_{i,\tilde{K},\mathcal{V}}^{\text{RO}}$ for derandomization in practice. For a string X , $\text{left}_l(X)$ returns the l most significant bits of L if $|L| > l$ and returns L otherwise.

Proposition 5. *For any BKSE scheme $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ that has the correctness property, the adversary $\mathcal{B}_{\text{drnd}}$ defined in Fig. 8 and the subversion $\tilde{\Pi} = (\tilde{\mathcal{K}}, \tilde{\mathcal{E}}, \tilde{\mathcal{D}})$ with $\tilde{\mathcal{K}} = \{0, 1\}^\kappa$ and $\tilde{\mathcal{E}}$ defined in Fig. 8, we have*

$$\text{Adv}_{\Pi, \tilde{\Pi}}^{\text{srv-lind}}(\mathcal{B}_{\text{drnd}}) = 1 - 2^{-l},$$

where $\mathcal{B}_{\text{drnd}}$ leaks l bits and makes a single ENC query and $\tilde{\Pi}$ meets the decryptability condition relative to Π .

Proof. Both the subverted encryption algorithm in the first ENC query, and the leakage function derive the same coins $r \leftarrow \text{RO}(\langle i, \tilde{K}, 0 \rangle, |r|)$. They also use the same big key, state and associated data when executing the original big-key encryption algorithm. Fixing these inputs turns $\mathcal{E}(K, \cdot, A, \sigma; r)$ into a deterministic injective mapping (otherwise Π would be incorrect). Therefore, the result of the leakage function L and the output of the RO applied to $\langle C \rangle$ are different strings with probability $1 - 2^{-l}$. The decryptability is trivial, as the subversion internally uses the actual encryption algorithm. $\square$

DISCUSSION. There are two possible interpretations of this result. Either the model captures a too strong goal and the derandomization attack has no meaningful implications, or thwarting simultaneous ASAs and KE is impossible without additional tools. We argue that the latter is true. The SURV-LIND notion formalizes a very intuitive and rather modest security goal: to protect confidentiality when faced with Big Brother’s tools. In addition, the derandomization attack can easily be translated into the following real-world attack.

DERANDOMIZATION IN PRACTICE. Consider a scenario where a high-value secret message is transmitted in a BKSE ciphertext at the beginning of a communication session. This could be e.g. a user ID sent to an online service during login. The target of the attack is then to identify logins with IDs belonging to a set of “tracked” users $\mathcal{V} \subset \mathcal{W}$, a subset of all user IDs in the system $\mathcal{W}$. Assume the set $\mathcal{V}$ has a canonical ordering $\mathcal{V} = \{V_1, \dots, V_u\}$.

Big Brother can mount a variant of the derandomization attack to achieve this, using the leakage function $\text{Lk}_{\tilde{K}, \mathcal{V}}$ described in Fig. 9. The function uses $\text{left}_l(L)$ instead of a query to the random oracle to return only the allowed l bits from the leakage function. We opt for this approach because in practice the output of any “decent” big-key encryption scheme can be truncated to the l most significant bits with a similar advantage compared to RO-hashing. The leakage function proceeds as follows. A secret V is easily recovered; after capturing a ciphertext C encrypting a V -dependent login, Big Brother partitions the leakage L into $C_1, \dots, C_w$ and tries to find an i s.t. $C = C_i$. If such i exists, then $V = V_i$, otherwise $V \notin \mathcal{V}$ or the attack fails. If $|L| \leq l$, the attack always succeeds. Otherwise, the probability of a false $V \notin \mathcal{V}$ conclusion is $1 - \lfloor l/|L| \rfloor$. With a leakage of 1 GB, size of an individual ciphertext ≤ 4 KB and success probability 1, 2^{18} users can be identified. Detecting the attack would be difficult, provided the users can only observe the inputs and outputs of the encryption algorithm, as the master key $\tilde{K}$ is used as a high-entropy seed for the generation of coins.

4 Resisting Algorithm Substitution Attacks and Key Exfiltration

We ask: what key property enables the devastating derandomization attack? The answer is the subversion’s ability to control the random coins, which allows the leakage function to perform useful computations in advance.

One countermeasure is the use of reverse cryptographic firewalls (RFs) [24, 38]. However, using an RF implicitly assumes trust in its *implementation*, which is problematic—especially for individual users who cannot verify that the RF itself has not been subverted.

Instead, we propose extending the setup with an external source of randomness, forcing the subversion to use its outputs. In other words, we add a new assumption to overcome the negative result of the previous section: a honest source of incorruptible randomness is available to both sender and receiver. In this section we show how, given this additional assumption, we can defeat ASAs and key exfiltration when incorporated into a new big-key symmetric encryption scheme. Our security model explicitly includes this honest randomness source, whose instantiation we describe in Section 6.

4.1 BKSE with Sessions

As we will show later, outsourcing randomness generation to a trusted external source (one observable by both the encrypting and decrypting parties) prevents the derandomization attack. However, accessing incorruptible random coins is expensive, making it impractical to use such randomness for the encryption of *every* message—as done in the BKSE construction of BKR.

At the same time, the big key should remain usable over an extended period, since exchanging a symmetric key of ≈ 1 TB is costly. We address this by augmenting BKSE communication sessions: the expensive external randomness is

used only to establish a session, after which all messages are processed by other means. This approach is efficient and enables reuse of the large symmetric key.

SYNTAX. We shift to an API-driven variant of symmetric encryption's syntax. Both the encryption and decryption are now a collection of algorithms (or subroutines) which can pass a state to one another. A BKSE scheme with sessions (BKSS) is a triple $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ with $\mathcal{K} = \{0, 1\}^k$ the big-key space. We define the message space $\mathcal{M} \subseteq \{0, 1\}^*$, the ciphertext space $\mathcal{C} \subseteq \{0, 1\}^*$, the associated data space $\mathcal{A} \subseteq \{0, 1\}^*$, the space of random seeds $\mathcal{R} \subseteq \{0, 1\}^*$ and the state space $\mathcal{S} \subseteq \{0, 1\}^*$. We require that $\forall M \in \mathcal{M} : M \in \{0, 1\}^m \Rightarrow \{0, 1\}^m \subseteq \mathcal{M}$.

The collection of encryption algorithms $\mathcal{E}$ contains two subroutines $\mathcal{E}.\text{Init}$ and $\mathcal{E}.\text{Enc}$. The possibly random and stateful algorithm $\mathcal{E}.\text{Init} : \mathcal{K} \times \mathcal{R} \times \mathcal{S} \rightarrow \mathcal{S}$ maps a big-key, a random seed and the current state $(\mathbf{K}, R, \sigma)$ to an updated state $\sigma' \leftarrow \mathcal{E}.\text{Init}(\mathbf{K}, R, \sigma)$. Its role is to set up a session (e.g. derive a conventional symmetric key) and store the setup in the state σ . The seed R is intended to be obtained from an external source of randomness. There is no other output value than the updated state; the decrypting party is assumed to be using the same external source of randomness. The possibly random and possibly stateful algorithm $\mathcal{E}.\text{Enc} : \mathcal{K} \times \mathcal{M} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathcal{C} \times \mathcal{S}$ maps a big-key, a plaintext, associated data and the current state $(\mathbf{K}, M, A, \sigma)$ to a ciphertext and an updated state $(C, \sigma') \leftarrow \mathcal{E}.\text{Enc}(\mathbf{K}, M, A, \sigma)$. Its role is to encrypt a messages in the session. We require that $\forall (\mathbf{K}, M, A, \sigma) \in (\mathcal{K}, \mathcal{M}, \mathcal{A}, \mathcal{S}) : (C, \sigma') \leftarrow \mathcal{E}.\text{Enc}(\mathbf{K}, M, A, \sigma) \Rightarrow |C| = |M| + \mu$, with $\mu \geq 0$ the stretch of Π .

The collection of decryption algorithms $\mathcal{D}$ contains two subroutines $\mathcal{D}.\text{Init}$ and $\mathcal{D}.\text{Dec}$. The deterministic and possibly stateful algorithm $\mathcal{D}.\text{Init} : \mathcal{K} \times \mathcal{R} \times \mathcal{S} \rightarrow \mathcal{S}$ maps a big-key, a random seed and the current state $(\mathbf{K}, R, \tau)$ to a state $\tau' \leftarrow \mathcal{D}.\text{Init}(\mathbf{K}, R, \tau)$. The deterministic and possibly stateful algorithm $\mathcal{D}.\text{Dec} : \mathcal{K} \times \mathcal{C} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathcal{M} \cup \{\perp\} \times \mathcal{S}$ maps a big-key, a ciphertext, associated data and the current state $(\mathbf{K}, C, A, \tau)$ to a recovered plaintext and an updated state $(M, \tau') \leftarrow \mathcal{D}.\text{Dec}(\mathbf{K}, C, A, \tau)$.

CORRECTNESS. The algorithms $\mathcal{E}.\text{Init}$ and $\mathcal{D}.\text{Init}$ are assumed to be executed synchronously, i.e., they should always use the same externally-generated seed. This assumption is embedded in the correctness condition; when the encrypting party encrypts a sequence of plaintexts partitioned into sessions and the decrypting party decrypts the resulting sequence of ciphertexts in order, partitioned in the same way and using the same seeds, the decrypting party will get the same sequence of plaintexts. Definition 8 formalizes this concept.

Definition 8 (Correctness of BKSS). A scheme $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ is correct if

$$\begin{aligned} & \forall \mathbf{K} \in \mathcal{K}, \forall q \in \mathbb{N}_{>0}, \forall (R_1, \dots, R_q) \in \mathcal{R}^q, \forall \vec{q} \in \mathbb{N}^q, \\ & \forall ((M_{1,1}, \dots, M_{1,\vec{q}[1]}), \dots, (M_{1,q}, \dots, M_{1,\vec{q}[q]})) \in \mathcal{M}^{\vec{q}[1]} \times \dots \times \mathcal{M}^{\vec{q}[q]}, \\ & \forall ((A_{1,1}, \dots, A_{1,\vec{q}[1]}), \dots, (A_{1,q}, \dots, A_{1,\vec{q}[q]})) \in \mathcal{A}^{\vec{q}[1]} \times \dots \times \mathcal{A}^{\vec{q}[q]} \end{aligned}$$

Game $\text{CORR}_{\Pi} \left(\mathbf{K}, q, R_1^q, \vec{q}, \left(M_{i,1}^{i, \vec{q}[i]} \right)_{i \in [1..q]}, \left(A_{i,1}^{i, \vec{q}[i]} \right)_{i \in [1..q]} \right)$ $\sigma, \tau \leftarrow \perp$ for $i \leftarrow 1, \dots, q$ do $\sigma \leftarrow \mathcal{E}.\text{Init}(\mathbf{K}, R_i, \sigma)$ $\tau \leftarrow \mathcal{D}.\text{Init}(\mathbf{K}, R_i, \tau)$ for $j \leftarrow 1, \dots, \vec{q}[i]$ do $(C_{i,j}, \sigma) \leftarrow \mathcal{E}.\text{Enc}(\mathbf{K}, M_{i,j}, A_{i,j}, \sigma)$ $(M'_{i,j}, \tau) \leftarrow \mathcal{D}.\text{Dec}(\mathbf{K}, C_{i,j}, A_{i,j}, \tau)$ return $(\forall i, j: C_{i,j} \neq \perp) \wedge (\exists i, j: M_{i,j} \neq M'_{i,j})$

Fig. 10: Game used to define correctness of a BKSS scheme Π . We use the notation $R_1^q = (R_1, \dots, R_q)$ and $M_{i,1}^{i, \vec{q}[i]} = (M_{i,1}, \dots, M_{i, \vec{q}[i]})$ for compactness.

the CORR_{Π} game shown in Fig. 10 returns 1 with probability zero.

SUBVERSIONS. A subversion of a BKSS scheme Π is a triple $\tilde{\Pi} = (\tilde{\mathcal{K}}, \tilde{\mathcal{E}}, \tilde{\mathcal{D}})$. $\tilde{\mathcal{K}}$, the space of (conventional) master-keys, is a finite nonempty set.

The collection of subverted encryption algorithms $\tilde{\mathcal{E}}$ contains two subroutines $\tilde{\mathcal{E}}.\text{Init}$ and $\tilde{\mathcal{E}}.\text{Enc}$. The subverted initialization subroutine $\tilde{\mathcal{E}}.\text{Init} : \tilde{\mathcal{K}} \times \mathcal{K} \times \mathcal{R} \times \tilde{\mathcal{S}} \times \mathcal{I} \rightarrow \tilde{\mathcal{S}}$ is a possibly randomized and stateful algorithm that maps a six-tuple of strings $(\tilde{K}, \mathbf{K}, R, \sigma, \iota)$ to an updated state σ' . The input ι represents some public information from a set of public user-identifiers $\mathcal{I}$. Note that the state space $\tilde{\mathcal{S}}$ of the subversion may differ from the original state space $\mathcal{S}$. The possibly randomized and stateful subverted encryption subroutine $\tilde{\mathcal{E}}.\text{Enc} : \tilde{\mathcal{K}} \times \mathcal{K} \times \mathcal{M} \times \mathcal{A} \times \tilde{\mathcal{S}} \times \mathcal{I} \rightarrow \mathcal{C} \times \tilde{\mathcal{S}}$ maps a tuple of strings $(\tilde{K}, \mathbf{K}, M, A, \sigma, i)$ to a ciphertext and an updated state (C, σ') .

The deterministic plaintext-recovery algorithm $\tilde{\mathcal{D}} : \tilde{\mathcal{K}} \times (\mathcal{C}^*)^* \times (\mathcal{A}^*)^* \times \mathcal{R}^* \times \mathcal{I} \rightarrow \mathcal{M}^* \cup \{\perp\}$ now works over vectors of vectors of ciphertexts and of associated data, because the grouping of ciphertexts into sessions can be meaningful for $\tilde{\mathcal{D}}$. It also accepts a vector of random seeds. The goal of the algorithm is to produce a vector of recovered plaintexts. We do not study the plaintext recovery any further, as our main goal is to rule out existence of *any* such $\tilde{\mathcal{D}}$.

DECRYPTABILITY. We define a BKSS variant of BPR's decryptability condition (Section 2.1). A subversion $\tilde{\Pi} = (\tilde{\mathcal{K}}, \tilde{\mathcal{E}}, \tilde{\mathcal{D}})$ is decryptable relative to the original scheme $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ if $(\tilde{K} \times \mathbf{K}, \tilde{\mathcal{E}}, \mathcal{D}')$ is a correct BKSS scheme, where $\mathcal{D}'.\text{Init}((\tilde{K}, \mathbf{K}), R, \tau) = \mathcal{D}.\text{Init}(\mathbf{K}, R, \tau)$ and $\mathcal{D}'.\text{Dec}((\tilde{K}, \mathbf{K}), C, A, \sigma) = \mathcal{D}.\text{Dec}(\mathbf{K}, C, A, \sigma)$.

SECURITY NOTION. The security notion RESIST (Definition 9, Fig. 11) formalizes the confidentiality of a BKSS scheme used with *external*, uncontestable randomness in the presence of simultaneous ASAs and key exfiltration in the ROM.

Our goal is to relate the (in)security of a BKSS scheme to that of a randomness-generation protocol and an encryption scheme Π . For now, we abstract away the details of the randomness-generation protocol and instead sample honest randomness; we will discuss how to instantiate such a protocol in Section 6.

In the RESIST game (Fig. 11), Big Brother $\mathcal{B}$ receives the master key $\tilde{K}$ and has access to the random oracle RO. It first outputs a leakage function Lk and a state τ . Then, $\mathcal{B}$ inputs τ and may query: (1) the LEAK oracle to obtain leakage on (possibly multiple) big keys; (2) the INIT oracle to initiate sessions; (3) the ENC oracle to make real-or-random (RoR) encryption queries. Finally, $\mathcal{B}$ outputs a guess b' .

When $\mathcal{B}$ makes an INIT query, another instance of $\mathcal{B}$ is run to tamper with the protocol that generates the seed for that session. For now, we assume this protocol samples honest randomness and is beyond adversarial control. Later, we show how to ensure incorruptible randomness even when the adversary tampers with the protocol.

To model this, the GENERATE oracle returns a randomness value R , which may be $\perp$ to reflect adversarial tampering. We assume adversarial tampering at this stage already to generalize our approach and because in Section 6 we show how to achieve uncontestable randomness generation using a protocol that the adversary can tamper with. If $\mathcal{B}$ is caught tampering with the protocol, the game aborts and $\mathcal{B}$ loses.

This introduces a technical complication: defining the advantage in the usual way (as in IND-CPA in Section 2) is insufficient, because the RESIST game has three possible outcomes: 0, 1, and **abort**. To address this, we adopt the advantage function from Vaudenay [52], which elegantly accounts for all outcomes: the advantage is small either if $\mathcal{B}$ causes too many aborts or if it performs poorly at guessing b when the game proceeds normally.

Definition 9. Let $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ be a BKSS scheme and $\tilde{\Pi} = (\tilde{\mathcal{K}}, \tilde{\mathcal{E}}, \tilde{\mathcal{D}})$ a corresponding subversion. Let $\mathcal{B}$ be an adversary playing RESIST, shown Fig. 11. $\mathcal{B}$'s advantage in winning the game is defined as

$$\mathbf{Adv}_{\Pi, \tilde{\Pi}}^{\text{resist}}(\mathcal{B}) = \Pr[\text{RESIST}_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow 1] - \Pr[\text{RESIST}_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow 0].$$

We say that a BKSS scheme Π resists simultaneous ASAs and KE if the advantage $\mathbf{Adv}_{\Pi, \tilde{\Pi}}^{\text{resist}}(\mathcal{B})$ is small for all adversaries with reasonable resources and any subversion $\tilde{\Pi}$ that is decryptable with respect to Π .

DISCUSSION. The game RESIST covers both the BKSS security and generation of external randomness, so the adversarial advantage covers all attack vectors of Big Brother. We purposefully avoid to specify the source of randomness at this stage to maintain modularity. We will explore later in this work how to instantiate the scheme with a concrete source of randomness. If the RESIST advantage is provably low for a scheme Π , then Π paired with a protocol that provides incorruptible randomness will resist to any leakage function and any subversion. As BKR, we assume that an APT exfiltrates the leakage *before* all of the communication takes place. This assumption is shared with BKR.

Game $\text{RESIST}_{\Pi, \tilde{H}}^{\mathcal{B}}$	Oracle $\text{ENC}(M, A, i)$
$\vec{K}_i, \sigma_i \leftarrow \perp$ everywhere	if $(\sigma_i = \perp)$ then return $\perp$
$\tilde{K} \leftarrow \$ \tilde{\mathcal{K}}$	$C_0, \sigma_i \leftarrow \$ \tilde{\mathcal{E}}.\text{Enc}^{\text{RO}}(\tilde{K}, \mathbf{K}_i, M, A, \sigma_i, i)$
$(\text{Lk}, \tau) \leftarrow \$ \mathcal{B}^{\text{RO}}(\tilde{K})$	$C_1 \leftarrow \$ \{0, 1\}^{ C_0 }$
$b \leftarrow \$ \{0, 1\}$	return C_b
$b' \leftarrow \$ \mathcal{B}^{\text{LEAK, INIT, ENC, RO}}(\tilde{K}, \tau)$	Oracle $\text{INIT}(i)$
return $(b = b')$	if $(\mathbf{K}_i = \perp)$ then return $\perp$
Oracle $\text{LEAK}(i)$	$R \leftarrow \$ \text{GENERATE}()$
if $(\mathbf{K}_i \neq \perp)$ then return $\perp$	if $R = \perp$ then abort
$\mathbf{K}_i \leftarrow \$ \mathcal{K}$	$\sigma_i \leftarrow \$ \tilde{\mathcal{E}}.\text{Init}^{\text{RO}}(\tilde{K}, \mathbf{K}_i, R, \sigma_i, i)$
$L \leftarrow \$ \text{Lk}^{\text{RO}}(\mathbf{K}_i)$	return R
return L	Procedure $\text{GENERATE}()$
	$R \leftarrow \$ \mathcal{R}$
	return R

Fig. 11: The game RESIST.

5 A BKSS Construction

We design a BKSS construction SES (for symmetric encryption with sessions). SES combines a symmetric encryption scheme with unique ciphertexts with BKR’s key encapsulation algorithm XKEY in a way that is provably RESIST-secure. The high-level intuition is simple: XKEY is used with a honest externally-generated seed to derive a conventional symmetric key, and a symmetric encryption scheme with unique ciphertexts is used with this key to secure a session.

THE SCHEME. SES takes four parameters: a conventional symmetric encryption scheme $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ with $\mathcal{K} = \{0, 1\}^\kappa$, and positive integers k, P, h . The BKSS scheme $\text{SES}[\Pi, k, P] = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ has $\mathcal{K} = \{0, 1\}^k$ with k a “big” power of two, and the encryption and decryption subroutines as defined in Fig. 12.

RESIST SECURITY OF SES. Theorem 1 shows that SES teamed-up with an incorruptible source of honest randomness is provably resistant to simultaneous ASA and KE. We refer by “queries made in the first stage” to the queries made by $\mathcal{B}$ to RO before it outputs the leakage function Lk and all queries made by the leakage function Lk to those oracle during all the LEAK queries. The “queries made in the second stage” are all the subsequent queries $\mathcal{B}$ makes to RO .

In the following, we let the term “session” refer to a $\mathcal{B}$ ’s $\text{INIT}(\iota, \hat{\tau})$ query and all subsequent $\text{ENC}(M, A, \iota)$ queries with the same identifier ι , until another INIT query is made with the same ι ; j^{th} session in the RESIST game means the session started by j^{th} INIT query made by $\mathcal{B}$.

Algorithm $\mathcal{E}.\text{Init}^{\text{RO}}(\mathbf{K}, R, \sigma)$ $K \leftarrow \text{XKEY}_{k,\kappa,P}^{\text{RO}}(\mathbf{K}, R)$ $\sigma \leftarrow K \parallel \varepsilon$ return σ	Algorithm $\mathcal{D}.\text{Init}^{\text{RO}}(\mathbf{K}, R, \tau)$ $K \leftarrow \text{XKEY}_{k,\kappa,P}^{\text{RO}}(\mathbf{K}, R)$ $\tau \leftarrow K \parallel \varepsilon$ return τ
Algorithm $\mathcal{E}.\text{Enc}^{\text{RO}}(\mathbf{K}, M, A, \sigma)$ parse $K \parallel \bar{\sigma} \leftarrow \sigma$ $C, \bar{\sigma} \leftarrow \mathcal{E}(K, M, A, \bar{\sigma})$ $\sigma \leftarrow K \parallel \bar{\sigma}$ return C, σ	Algorithm $\mathcal{D}.\text{Dec}^{\text{RO}}(\mathbf{K}, C, A, \tau)$ parse $K \parallel \bar{\tau} \leftarrow \tau$ $M, \bar{\tau} \leftarrow \mathcal{D}(K, C, A, \bar{\tau})$ $\tau \leftarrow K \parallel \bar{\tau}$ return M, τ
Algorithm $\text{XKEY}_{k,\kappa,P}^{\text{RO}}(\mathbf{K}, R)$ for $i \leftarrow 1, \dots, P$ do $\vec{P}[i] \leftarrow \text{RO}(\langle R, i, 0 \rangle, \log_2(k))$ $J \leftarrow \mathbf{K}[\vec{P}]$ $K \leftarrow \text{RO}(\langle R, J, 1 \rangle, \kappa)$ return K	

Fig. 12: The BKSS scheme $\text{SES}[\Pi, k, P] = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ with $\mathcal{K} = \{0, 1\}^k$ and $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ be a conventional symmetric encryption scheme with unique ciphertexts, such that $\mathcal{K} = \{0, 1\}^\kappa$.

Theorem 1. *Let $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ be an encryption scheme with unique ciphertexts with $\mathcal{K} = \{0, 1\}^\kappa$. Let k, P, h be positive integers. Let $\Pi = \text{SES}[\Pi, k, P]$ and Let $\tilde{\Pi} = (\tilde{\mathcal{K}}, \tilde{\mathcal{E}}, \tilde{\mathcal{D}})$ be a subversion of Π that meets the decryptability condition. Let $\mathcal{B}$ be an adversary. Then*

$$\text{Adv}_{\Pi, \tilde{\Pi}, h, p, \ell}^{\text{resist}}(\mathcal{B}) \leq \Delta_1 + \Delta_2 + \Delta_3,$$

with $\Delta_1 = 0$, $\Delta_2 = 2 \cdot q_K \cdot \text{Adv}_{\text{XKEY}}^{\text{ukey}}(\mathcal{A})$, $\Delta_3 = 2 \cdot q_I \cdot \text{Adv}_{\Pi}^{\text{ind\$}}(\mathcal{A}')$, if $\mathcal{B}$ leaks at most l bits, makes at most q_K LEAK queries, at most q_I INIT queries, makes no more than q_E ENC queries which process no more than d bits of data in a session, makes at most q_1 and q_2 queries to RO in the first and the second stage respectively, such that $q = q_1 + q_2$, and runs in time t . Let $r = (q + q_K + q_I + q_E + d)$.

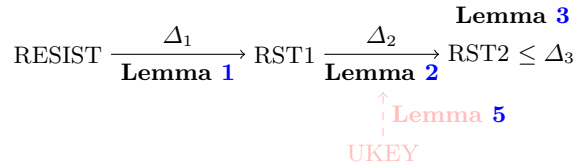


Fig. 13: An overview of the proofs of Theorem 1 and Theorem 2.

The UKEY adversary $\mathcal{A}$ makes at most q_K DERIVE queries and runs in time $t + \gamma \cdot r$ for a positive constant γ . The IND $\mathcal{S}$ -CPA adversary $\mathcal{A}'$ makes at most q_E encryption queries with total data bounded by d bits and runs in time $t + \gamma' \cdot r$ for a positive constant γ' .

Proof. The proof is based on a hybrid argument, with hybrids RST1 and RST2 defined in Fig. 14. The high level overview of the proof is depicted in Fig. 13. Setting RST0 = RESIST, we have

$$\begin{aligned} \text{Adv}_{\Pi, \tilde{\Pi}}^{\text{resist}}(\mathcal{B}) &= \Pr[\text{RST2}_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow 1] - \Pr[\text{RST2}_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow 0] \\ &+ \sum_{i=1}^2 \sum_{\beta=0}^1 (-1)^{1-\beta} (\Pr[\text{RST}(i-1)_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow \beta] - \Pr[\text{RST}i_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow \beta]). \end{aligned}$$

Theorem 2 follows from triangular inequality after upper-bounding the inner sum for $i = 1$ (Lemma 1 yields Δ_1) and for $i = 2$ (Lemma 2 yields Δ_2), and upper-bounding the term $\Pr[\text{RST2}_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow 1] - \Pr[\text{RST2}_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow 0]$ (Lemma 3 yields Δ_3). $\square$

Game $\text{RST1}_{\Pi, \tilde{\Pi}}^{\mathcal{B}}, \boxed{\text{RST2}_{\Pi, \tilde{\Pi}}^{\mathcal{B}}}$	Oracle $\text{ENC}(M, A, i)$
$\vec{K}_i, \sigma_i \leftarrow \perp$ everywhere	if $(\sigma_i = \perp)$ then return $\perp$
$\tilde{K} \leftarrow \mathcal{K}$	$C_0, \sigma_i \leftarrow \mathcal{E}.\text{Enc}^{\text{RO}}(\tilde{K}_i, M, A, \sigma_i)$
$(\text{Lk}, \tau) \leftarrow \mathcal{B}^{\text{RO}}(\tilde{K})$	$C_1 \leftarrow \{0, 1\}^{ C_0 }$
$b \leftarrow \{0, 1\}$	return C
$b' \leftarrow \mathcal{B}^{\text{LEAK, INIT, ENC, RO}}(\tilde{K}, \tau)$	Oracle $\text{INIT}(i, \hat{\tau})$
return $(b = b')$	if $(K_i = \perp)$ then return $\perp$
Oracle $\text{LEAK}(i)$	$\hat{\tau}, R \leftarrow \text{GENERATE}(\hat{\tau})$
if $(K_i \neq \perp)$ then return $\perp$	if $(R = \perp)$ then abort
$K_i \leftarrow \mathcal{K}$	$\sigma_i \leftarrow \mathcal{E}.\text{Init}^{\text{RO}}(K_i, R, \sigma_i)$
$L \leftarrow \text{Lk}^{\text{RO}}(K_i)$	$\boxed{K \leftarrow \{0, 1\}^{\kappa}; \sigma_i \leftarrow K \parallel \varepsilon}$
return L	return $\hat{\tau}, R$
	Procedure $\text{GENERATE}(\hat{\tau})$
	$R \leftarrow \mathcal{R}$
	return $\hat{\tau}, R$

Fig. 14: The games RST1 (without the boxed statement) and $\boxed{\text{RST2}}$. The lines that differ from RESIST are marked with in grey. In RST1, we replace $\tilde{\Pi}$ by the original scheme Π . In RST2, we additionally sample the derived keys uniformly.

Lemma 1. *Let Π be an encryption scheme with unique ciphertexts with $\mathcal{K} = \{0, 1\}^\kappa$. Let k and P be positive integers. Let $\mathbf{\Pi} = \text{SES}[\Pi, k, P]$. Let $\tilde{\mathbf{\Pi}} = (\tilde{\mathcal{K}}, \tilde{\mathcal{E}}, \tilde{\mathcal{D}})$ be a subversion of $\mathbf{\Pi}$ that meets the decryptability condition. Then for any adversary $\mathcal{B}$ and any $\beta \in \{0, 1\}$ we have that*

$$\Pr[\text{RESIST}_{\mathbf{\Pi}, \tilde{\mathbf{\Pi}}}^{\mathcal{B}} \Rightarrow \beta] = \Pr[\text{RST1}_{\mathbf{\Pi}, \tilde{\mathbf{\Pi}}}^{\mathcal{B}} \Rightarrow \beta],$$

which implies that $\Delta_1 = 0$.

Proof. The proof shows that a subversion of $\text{SES}[\Pi, k, P]$ for which the decryptability condition holds always outputs the same ciphertexts as the original scheme $\text{SES}[\Pi, k, P]$ would, in any possible sequence of sessions. We present the full proof in Section B.2. $\square$

We use the incorruptible randomness R to seed XKEY, a setting that is not treated by BKR. We define the notion UKEY (Definition 10) that captures the security of big-key encapsulation paired with an incorruptible-randomness protocol, that, as before, we abstract away for now by sampling randomness directly in GENERATE. Note that UKEY aborts if $\mathcal{A}$ is caught cheating.

Definition 10. *Let $\mathcal{S}$ be a big-key encapsulation scheme. Let $\mathcal{A}$ be an adversary playing UKEY (Fig. 15). $\mathcal{A}$'s advantage in winning the game is defined as*

$$\text{Adv}_{\mathcal{S}}^{\text{ukey}}(\mathcal{A}) = \Pr[\text{UKEY}_{\mathcal{S}}^{\mathcal{A}} \Rightarrow 1] - \Pr[\text{UKEY}_{\mathcal{S}}^{\mathcal{A}} \Rightarrow 0].$$

Lemma 2. *Let $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ be an encryption scheme with unique ciphertexts with $\mathcal{K} = \{0, 1\}^\kappa$. Let k and P be positive integers. Let $\mathbf{\Pi} = \text{SES}[\Pi, k, P]$. Assume that the Let $\mathcal{B}$ be an adversary that leaks at most l bits, makes at most q_k LEAK queries, at most q_I INIT queries, no more than q_E ENC queries which process no more than d bits of data in a session, makes at most q_1 and q_2 queries to the oracles in $\mathcal{O}$ in the first and the second stage respectively, with $q = q_1 + q_2$, has computational resources limited by c and runs in time t . Then*

$$\left| \sum_{\beta=0}^1 (-1)^{1-\beta} (\Pr[\text{RST1}_{\mathbf{\Pi}, \tilde{\mathbf{\Pi}}}^{\mathcal{B}} \Rightarrow \beta] - \Pr[\text{RST2}_{\mathbf{\Pi}, \tilde{\mathbf{\Pi}}}^{\mathcal{B}} \Rightarrow \beta]) \right| \leq 2 \cdot q_k \cdot \text{Adv}_{\text{XKEY}_{k, \kappa, P}}^{\text{ukey}}(\mathcal{A})$$

for an adversary $\mathcal{A}$ that leaks at most l bits, makes no more than q_I DERIVE queries, and makes no more than q_1 and q_2 queries to the oracles in $\mathcal{O}$ in the first and the second phase respectively.

Proof. The proof is based on a hybrid argument. Assuming that $\mathcal{B}$ queries the LEAK oracle with q_k identifiers, we define $q_k + 1$ hybrids $\text{RST1}.j$ for $j = 0, \dots, q_k$. In the game $\text{RST1}.j$, the queries $\text{INIT}(\hat{\tau}, \iota_u)$ are processed as in RST2 (the derived key K is uniform) for $1 \leq u \leq j$, while for $j < u \leq q_k$ they are processed as in RST1 (K is derived using XKEY). We present the complete proof in Section B.3. $\square$

Lemma 3. *Let $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ be an encryption scheme with unique ciphertexts with $\mathcal{K} = \kappa$. Let k and P be positive integers. Let $\mathbf{\Pi} = \text{SES}[\Pi, k, P]$. Let $\mathcal{B}$, be an*

Game $\text{UKEY}_{\mathcal{S}}^{\mathcal{O}}$	Oracle DERIVE	Procedure $\text{GENERATE}()$
$(\text{Lk}, \sigma) \leftarrow \mathcal{A}^{\mathcal{O}}$	$R \leftarrow \mathcal{S} \text{ GENERATE}()$	$R \leftarrow \mathcal{R}$
$\mathbf{K} \leftarrow \mathcal{S} \{0, 1\}^k$	if $(R = \perp)$ then abort	return R
$L \leftarrow \text{Lk}^{\mathcal{O}}(\mathbf{K})$	if $(b = 0)$ then	
$b \leftarrow \mathcal{S} \{0, 1\}$	$K \leftarrow \mathcal{S} \{0, 1\}^{\kappa}$	
$b' \leftarrow \mathcal{A}^{\text{DERIVE}, \mathcal{O}}(L, \sigma)$	else	
return $(b = b')$	$K \leftarrow \mathcal{S}^{\text{RO}}(\mathbf{K}, R)$	
	return R, K	

Fig. 15: Game capturing security of big-key encapsulation with selectors generated by some source of incorruptible randomness, where $\mathcal{O} = \{\text{RO}, \text{RO}'\}$.

adversary playing game RST2 (Fig. 14) that leaks at most l bits, makes at most q_k LEAK queries, at most q_I INIT queries, makes no more than q_E ENC queries which process no more than d bits of data in a session, makes at most q_1 and q_2 queries to RO in the first and the second stage respectively, has computational resources limited by c and runs in time t . Then we have

$$\Pr[\text{RST2}_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow 1] - \Pr[\text{RST2}_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow 0] \leq 2 \cdot q_I \cdot \text{Adv}_{\Pi}^{\text{ind}\$}(\mathcal{A}),$$

for an $\text{IND}\$$ -CPA adversary $\mathcal{A}$ that makes at most q_E encryption queries with total data complexity bounded by d bits and runs in time $t + \gamma \cdot (q_1 + q_2 + q_K + q_I + q_E + d)$ for some positive constant γ .

Proof. The proof uses an hybrid argument on the q_I sessions that the adversary spawns by calling the INIT oracle. We show the full proof in Section B.4. $\square$

6 Uncontestable Random Number Generation

In this section, we show how to design a protocol that enables provably uncontestable random number generation, thus providing the final component needed to achieve security against coordinated algorithm substitution attacks and key exfiltration.

The first concept of a public randomness source was the *random beacon*, introduced by Rabin [43]. A random beacon is an online service that broadcasts unpredictable random values at regular intervals—e.g., the NIST randomness beacon [41]. The problem with such beacons is the lack of verifiability: we cannot tell whether they are secretly controlled by Big Brother and must simply trust them. This trust requirement is unacceptable when facing an adversary with the resources and influence to compromise such infrastructure.

Instead, we adopt a protocol by Lenstra and Wesolowski (LW) [36], called *unicorn*, which offers *provably uncontestable* randomness generation. The protocol is interactive and allows a variable number of parties to jointly sample a

common random value, with each participant able to verify that no tampering occurred—without trusting the others. We adapt *unicorn* to fit our security model by modifying its syntax, while preserving the underlying computation.

6.1 The *unicorn* Protocol

ALGORITHMS, PARTICIPANTS AND MESSAGES. In the rest of the section, we let $\mathfrak{P}$ denote the *unicorn* protocol. $\mathfrak{P}$ uses the following algorithms:

- A hash-based commitment algorithm $\mathfrak{P}.\text{commit}^H : \{0, 1\}^* \rightarrow \{0, 1\}^h$ that computes a commitment $c \in \{0, 1\}^h$ for a seed $s \in \{0, 1\}^*$, using a conventional hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^h$. commit must be either randomized or use a RO.
- A slow-timed, verifiable hash function $\mathfrak{P}.\text{SH}^{H,p,\ell,\theta} : \{0, 1\}^* \rightarrow \{0, 1\}^h \times \mathcal{W}$ that maps a seed $s \in \{0, 1\}^*$ to a fixed sized output $g \in \{0, 1\}^h$ and a witness from the set of witnesses $w \in \mathcal{W}$. The algorithm SH is parametrized by a conventional hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^h$, a prime number p with $p \geq 2^h$ and $p \equiv 3 \pmod{4}$, a positive integer ℓ , and a permutation $\theta : \mathbb{Z}_p^* \rightarrow \mathbb{Z}_p^*$.
- A verification algorithm $\mathfrak{P}.\text{ver}^{H,p,\ell,\theta} : \{0, 1\}^* \times \{0, 1\}^h \times \{0, 1\}^h \times \mathcal{W} \rightarrow \{0, 1\}$ that takes a seed $s \in \{0, 1\}^*$, a commitment $c \in \{0, 1\}^h$, a value $g \in \{0, 1\}^h$ and a witness $w \in \mathcal{W}$ and outputs a single bit that indicates if $c = \text{commit}(s)$ and $g, w = \text{SH}(s)$. The algorithm ver is parametrized by a conventional hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^h$, a prime number p with $\log_2(p) \geq h$, a positive integer ℓ , and a permutation $\theta : \mathbb{Z}_p \rightarrow \mathbb{Z}_p$.

The three algorithms used in *unicorn* are described in detail in Section C. We let $\mathfrak{P}^{H,p,\ell,\theta}$ denote the instance of the *unicorn* protocol parametrized by H, p, ℓ , and θ . We omit some of the parameters, if they are implicit from the context.

There are two types of participants in a $\mathfrak{P}$, a single participant P_0 that carries out the computation, and n seed-contributing participants $P_1, \dots, P_n$. An execution of a $\mathfrak{P}$ proceeds in three, time-limited phases.

The first is the collection phase, taking place between wall-clock times t_{-1} and t_0 , in which each participant sends their seed-contribution to P_0 . Next, in the commitment phase, taking place between times t_0 and t_1 , P_0 completes the seed with their own contribution, commits to the complete seed and broadcasts the commitment. Finally, in the generation phase which takes place between times t_1 and t_2 , P_0 computes the verifiable hash, and broadcasts it along with the witness. The exact execution of $\mathfrak{P}$ is depicted in Fig. 16.

SECURITY MODEL. LW model the worst case scenario, where all but one participants collude in their *incorruptibility* notion. We extend their notion by allowing the adversary to break the commitment. The corresponding security game, labelled INCORR, is defined in Fig. 17. The conventional hash function is modelled as a RO and the permutation θ as a random permutation. INCORR is parametrized by a time limit δ and a target function $t : \{0, 1\}^h \rightarrow \{0, 1\}$. The

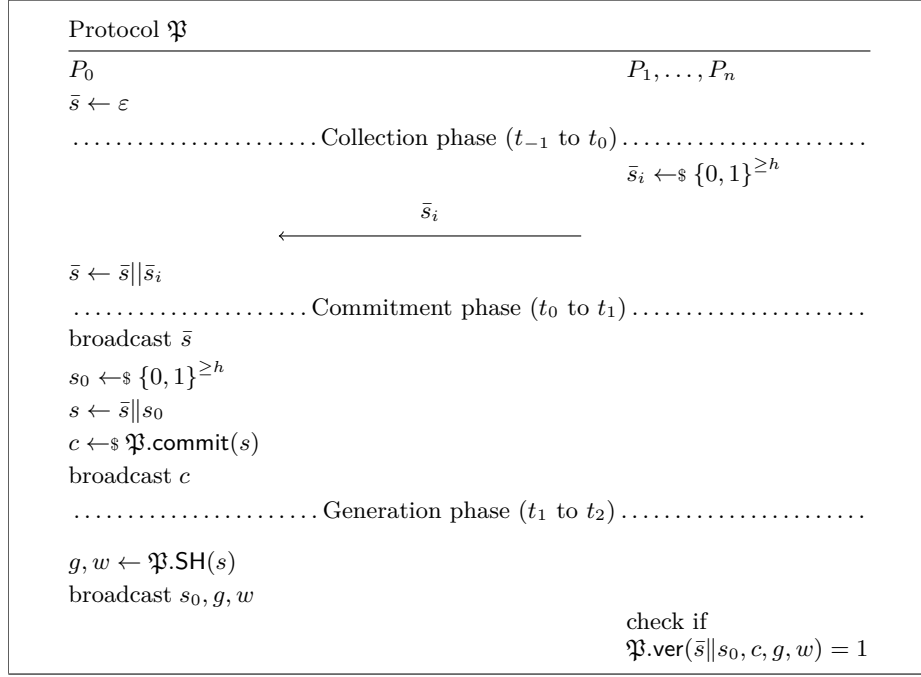
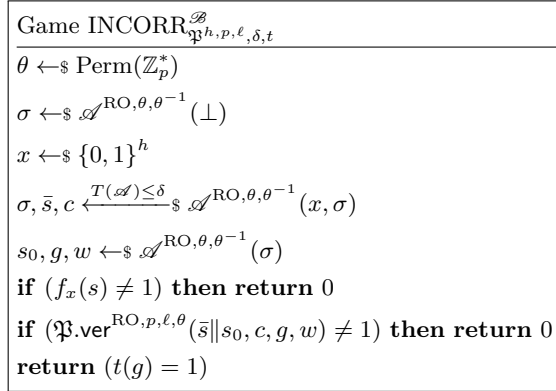
Fig. 16: An execution of the protocol $\mathfrak{P} = \text{unicorn}$.

Fig. 17: Game used to define incorruptibility of $\mathfrak{P} = \text{unicorn}$ protocol. INCORR is parametrized by a positive time limit δ and a target function $t : \{0, 1\}^h \rightarrow \{0, 1\}$. Here $s \xleftarrow{T(\mathcal{A}) \leq \delta} \$ \mathcal{A}^{\text{RO}, \theta, \theta^{-1}}(x)$ denotes running the adversary $\mathcal{A}$ with oracles RO, θ and θ^{-1} and input x for time at most δ to obtain the outputs $\sigma, \bar{s}, c$; if $\mathcal{A}$ does not terminate within the time limit, $\sigma, \bar{s}, c$ all become the empty string ε . The function $f_x : \{0, 1\}^* \rightarrow \{0, 1\}$ checks if the seed s contains x and is a valid seed string for $\mathfrak{P}$. It returns $f_x(s) = 1$ if and only if $s = s_0 \parallel x \parallel s_1$ with $|s_i| = 0$ or $|s_i| \geq h$ for $i = 1, 2$.

time limit δ models the time between the moment when the last honest seed contribution x appears in the protocol, and the time t_1 , when the commitment on the seed has been published and the computation of the value g starts. Before receiving x , the adversary is allowed to do precomputations. The target function t models any property the adversary $\mathcal{A}$ may want to force on the generated value g . The adversary wins the INCORR game if it can produce an incomplete seed $\bar{s}$ and a commitment c within time limit δ , such that it can later complete the seed and produce an output g and a witness that will comprise a “valid” instance of **unicorn** and such that $t(g) = 1$. The security of **unicorn** relies on the assumption that computing square roots modulo a prime p is about $\log(p)$ times slower than computing squares, without the possibility of parallelization. This is formalized in Definition 11. Iterative computation of modular square roots then yields a special “hash function” which is slow to evaluate but fast to invert.

Definition 11 ([36]). *Given a length $\Delta > 0$ of a time interval, a probability ϵ , and a bound on computational resources c , the (Δ, ϵ, c) -**sqrt**(p) assumption is the following: any party with computational resources bounded by c that gets a uniformly sampled $x \in QR_p$ at time t , succeeds to compute a $y \in \mathbb{Z}_p^*$ s.t. $y^2 \equiv x \pmod{p}$ before time $t + \Delta$ with probability at most ϵ .*

The intuition behind **unicorn**’s security is that allowing the participants to contribute to the input seed during an interval that is shorter than the time necessary to evaluate the slow hash ensures that no one can predict the output of the hash fast enough to influence it. We extend the result on the security of the **unicorn** protocol by LW in Lemma 4. With a slight abuse of notation, for a function $f : \mathcal{D} \rightarrow \mathcal{R}$ and an element $r \in \mathcal{R}$ we let $f^{-1}(r)$ denote the set $\{d \in \mathcal{D} \mid f(d) = r\}$. The high level intuition behind Lemma 4 is that unless the adversary breaks the hash-based commitment, it will not have enough time to find a suitable seed that will satisfy the target function $t(\cdot)$, because computing $\mathfrak{P}$.SH takes more time than δ with overwhelming probability.

Lemma 4. *Assume that the (Δ, ϵ, c) -**sqrt**(p) assumption holds. Then we have*

$$\Pr[\text{INCORR}_{\mathfrak{P}^{h,p,\ell},\delta,t}^{\mathcal{B}} \Rightarrow 1] \leq \frac{q+1}{2^h} + \frac{2q + |t^{-1}(1)|}{2^{h-1}} + \frac{q + \epsilon \ell q(p-1)}{p-1-q}$$

for any $\delta < \ell\Delta$ and any adversary $\mathcal{A}$ who makes at most q queries to all its oracles in total and whose computational resources are limited by c .

Proof. The proof of Lemma 4 is presented in Section C.2. $\square$

Corollary 1. *Given the conditions from Lemma 4, we have for the random variable g in the game $\text{INCORR}_{\mathfrak{P}^{p,\ell},\delta,t}^{\mathcal{B}}$ that $\Pr[g = \gamma] \leq \frac{q+1}{2^h} + \frac{2q+1}{2^{h-1}} + \frac{q+\epsilon\ell q(p-1)}{p-1-q}$ for any $\gamma \in \mathbb{Z}_p^*$.*

Proof. For every $x \in \mathbb{Z}_p^*$, we define a target function $t_x : \mathbb{Z}_p^* \rightarrow \{0, 1\}$ defined as $t_x(x') = 1$ iff $x = x'$. We thus have $\Pr[g = x] = \Pr[t_x(g) = 1]$ and because $|t_x^{-1}(1)| = 1$ for all $x \in \mathbb{Z}_p^*$, the corollary follows. $\square$

6.2 BKSS with unicorn

In the previous section we show how parties can use the `unicorn` protocol to produce uncontestable randomness. In this section we show that this uncontestable source of randomness implies RESIST security. To this end we first integrate the `unicorn` protocol into our security definitions, we then prove a concrete bound on the key encapsulation mechanism that the `unicorn`-generated randomness seeds and conclude with the theorem that states RESIST-security of our approach.

SECURITY NOTION. We adapt the RESIST notion (Definition 9, Fig. 11) into the security notion $\overline{\text{RESIST}}$ (Definition 12, Fig. 18) to capture the *entire setup* by embedding the security notion of the `unicorn` protocol directly into $\overline{\text{RESIST}}$. The $\overline{\text{RESIST}}$ game closely resembles the RESIST game, but the embedding `unicorn` introduces several modifications. When Big Brother $\mathcal{B}$ makes a INIT query, a second instance of $\mathcal{B}$ is invoked to tamper with the `unicorn` protocol responsible for generating the seed used in the INIT query. Accordingly, the GENERATE procedure now inputs and outputs a state $\hat{\tau}$, allowing Big Brother to interact with this second instance during seed generation. The model includes two independent random oracles, RO and RO', as well as a special random permutation θ and its inverse θ^{-1} . The use of two random oracles separates the BKSS scheme from the source of randomness⁵. The permutation θ is part of the `unicorn` protocol. We denote the collection of all these oracles by $\mathcal{O}$. The $\overline{\text{RESIST}}$ game is parameterized by positive integers h , ℓ , and a prime p , which are all parameters of `unicorn`.

Definition 12. Let $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ be a BKSS scheme and $\tilde{\Pi} = (\tilde{\mathcal{K}}, \tilde{\mathcal{E}}, \tilde{\mathcal{D}})$ a corresponding subversion. Let $\mathcal{B}$ be an adversary playing $\overline{\text{RESIST}}$, shown Fig. 18. $\mathcal{B}$'s advantage in winning the game is defined as

$$\text{Adv}_{\Pi, \tilde{\Pi}}^{\overline{\text{resist}}}(\mathcal{B}) = \Pr \left[\overline{\text{RESIST}}_{\Pi, \tilde{\Pi}, h, p, \ell}^{\mathcal{B}} \Rightarrow 1 \right] - \Pr \left[\overline{\text{RESIST}}_{\Pi, \tilde{\Pi}, h, p, \ell}^{\mathcal{B}} \Rightarrow 0 \right].$$

We say that a BKSS scheme Π paired with `unicorn` _{h, p, ℓ} resists simultaneous ASAs and KE if $\text{Adv}_{\Pi, \tilde{\Pi}, h, p, \ell}^{\overline{\text{resist}}}(\mathcal{B})$ is small for all adversaries with reasonable resources and any subversion $\tilde{\Pi}$ that is decryptable with respect to Π .

DISCUSSION. The game $\overline{\text{RESIST}}$ encompasses both BKSS security and randomness generation via `unicorn`, so the adversarial advantage accounts for all attack vectors available to Big Brother. Moreover, $\overline{\text{RESIST}}$ demonstrates how to instantiate the general RESIST notion with a concrete protocol for generating incorruptible randomness, illustrating the generality of our approach and enabling future work to adapt our notions to alternative sources of randomness. If the $\overline{\text{RESIST}}$ advantage is provably low for a scheme Π , then Π combined with `unicorn` resists any leakage function and subversion, provided the following assumptions hold. First, as with RESIST and as in BKR, we assume that an APT exfiltrates leakage *before* any communication occurs. Second, we adopt

⁵ This is equivalent to using a single RO with prefix domain separation.

Game $\overline{\text{RESIST}}_{\Pi, \tilde{\Pi}, h, p, \ell}^{\mathcal{B}}$	Oracle $\text{ENC}(M, A, i)$
$\vec{K}_i, \sigma_i \leftarrow \perp$ everywhere	if $(\sigma_i = \perp)$ then return $\perp$
$\tilde{K} \leftarrow \mathcal{K}$	$C_0, \sigma_i \leftarrow \mathcal{E}.\text{Enc}^{\mathcal{O}}(\tilde{K}, K_i, M, A, \sigma_i, i)$
$\theta \leftarrow \text{Perm}(\mathbb{Z}_p^*)$	$C_1 \leftarrow \{0, 1\}^{ C_0 }$
$(\text{Lk}, \tau) \leftarrow \mathcal{B}^{\mathcal{O}}(\tilde{K})$	return C_b
$b \leftarrow \{0, 1\}$	Oracle $\text{INIT}(i, \hat{\tau})$
$b' \leftarrow \mathcal{B}^{\text{LEAK}, \text{INIT}, \text{ENC}, \mathcal{O}}(\tilde{K}, \tau)$	if $(K_i = \perp)$ then return $\perp$
return $(b = b')$	$\hat{\tau}, R \leftarrow \text{GENERATE}(\hat{\tau})$
Oracle $\text{LEAK}(i)$	if $R = \perp$ then abort
if $(K_i \neq \perp)$ then return $\perp$	$\sigma_i \leftarrow \mathcal{E}.\text{Init}^{\mathcal{O}}(\tilde{K}, K_i, R, \sigma_i, i)$
$K_i \leftarrow \mathcal{K}$	return $\hat{\tau}, R$
$L \leftarrow \text{Lk}^{\mathcal{O}}(K_i)$	Procedure $\text{GENERATE}(\hat{\tau})$
return L	$x \leftarrow \mathcal{R}$
	$\hat{\tau}, \bar{s}, c \xleftarrow{T(\mathcal{B}) \leq \delta} \mathcal{B}^{\mathcal{O}}(x, \hat{\tau})$
	$\hat{\tau}, s_0, R, w \leftarrow \mathcal{B}^{\mathcal{O}}(\hat{\tau})$
	if $(f_x(s) \neq 1)$ then $R \leftarrow \perp$
	if $(\mathfrak{P}.\text{ver}^{\mathcal{O}', h, p, \ell}(\bar{s} \ s_0, c, R, w) \neq 1)$ then $R \leftarrow \perp$
	return $\hat{\tau}, R$

Fig. 18: The game $\overline{\text{RESIST}}$. We let $\mathfrak{P} = \text{unicorn}$. Because of the inclusion of the unicorn security notion, we fix $\mathcal{R} = \{0, 1\}^h$ for a positive h . We use $\mathcal{O}$ as a shorthand for $\text{RO}, \text{RO}', \theta, \theta^{-1}$ and $\mathcal{O}'$ as a shorthand for $\text{RO}', \theta, \theta^{-1}$.

the $(\Delta, \epsilon, c)\text{-sqrt}(p)$ assumption, which appears realistic for practical parameters and remains unchallenged since LW's 2015 paper [36]. Third, we assume that the user successfully contributes their seed to the unicorn protocol. This assumption is not trivial: relying on uncompromised software to upload the seed begs the question. However, the seed can be submitted in a human-verifiable way, making timely manipulation by Big Brother difficult [36]. Furthermore, it suffices for either the encrypting or decrypting party to upload their seed for security to hold. At the end of this section, we discuss other possible sources of randomness, whose formal treatment is left as future work.

RESIST SECURITY OF SES WITH unicorn. Theorem 2 shows that SES teamed-up with unicorn is provably resistant to simultaneous ASA and KE. The only missing piece from Theorem 2 is to bound the advantage of the key encapsulation mechanism which is now seeded by unicorn. We first state the theorem and then prove the concrete bound on the key encapsulation mechanism.

Theorem 2. Let $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ be an encryption scheme with unique ciphertexts with $\mathcal{K} = \{0, 1\}^\kappa$. Let k, P, h and ℓ be positive integers. Let p be a prime number with $p \equiv 3 \pmod{4}$ and $p \geq 2^h$. Let $\Pi = \text{SES}[\Pi, k, P, h]$. Let $\tilde{\Pi} = (\tilde{\mathcal{K}}, \tilde{\mathcal{E}}, \tilde{\mathcal{D}})$ be a subversion of Π that meets the decryptability condition. Let $\mathcal{B}$ be an adversary and $\overline{\text{RESIST}}$ the game in Fig. 18. Assume that the $(\Delta, \epsilon, c)\text{-sqr t}(p)$ assumption holds. Then

$$\text{Adv}_{\Pi, \tilde{\Pi}, h, p, \ell}^{\overline{\text{resist}}}(\mathcal{B}) \leq \Delta_2 + \Delta_3,$$

where

$$\Delta_2 = 2 \cdot q_k \cdot \left(q_2 \cdot q_I \cdot G_{k, p}(2^{k-l}) + (q_I(2q_1 + q_I - 1)) \cdot \left(\frac{q+1}{2^h} + \frac{2q+1}{2^{h-1}} + \frac{q+\epsilon\ell q(p-1)}{p-1-q} \right) \right),$$

$$\Delta_3 = 2 \cdot q_I \cdot \text{Adv}_{\Pi}^{\text{ind\$}}(\mathcal{A}),$$

if $\mathcal{B}$ leaks at most l bits, makes at most q_K LEAK queries, at most q_I INIT queries, makes no more than q_E ENC queries which process no more than d bits of data in a session, makes at most q_1 and q_2 queries to the oracles in $\mathcal{O}$ in the first and the second stage respectively, such that $q = q_1 + q_2$, has computational resources limited by c and runs in time t . The IND\$-CPA adversary $\mathcal{A}$ makes at most q_E encryption queries with total data bounded by d bits and runs in time $t + \gamma \cdot (q_1 + q_2 + q_K + q_I + q_E + d)$ for a positive constant γ .

The function $G_{k, P}$ is defined by BKR [13, Theorem 2] and it is tightly approximated by $G_{k, P}(2^{k-l}) \approx 2^{-P \cdot w(\frac{1}{k})}$, for $w(\lambda) = -\lg(1 - H_2^{-1}(1 - \lambda))$, where H_2 is the binary entropy function and $H_2^{-1}(1 - \lambda)$ is the smaller of the two possible inverses of $1 - \lambda$ under H_2 .

Proof. The high level overview of the proof is depicted in Fig. 13. The proof builds on Theorem 1 with the concrete bound for Δ_3 that Lemma 5 yields. $\square$

Lemma 5. Let k, κ, P, h and ℓ be positive integers, and δ, Δ be lengths of time intervals such that $\ell \cdot \Delta > \delta$. Let p be a prime number with $p \equiv 3 \pmod{4}$ and $p \geq 2^h$. Assume that the $(\Delta, \epsilon, c)\text{-sqr t}(p)$ assumption holds. Let $\mathcal{A}$ be an adversary that plays the UKEY game and that leaks at most l bits, makes q_I DERIVE queries, makes at most q_1 and q_2 queries to the oracles in $\mathcal{O}$ in the first and the second stage respectively, with $q = q_1 + q_2$, and has computational resources limited by c . Then

$$\text{Adv}_{\text{XKEY}_{k, \kappa, P, h, h, p, \ell}}^{\text{ukey}}(\mathcal{A}) \leq \left(q_2 \cdot q_I \cdot G_{k, p}(2^{k-l}) + (q_I(2q_1 + q_I - 1)) \cdot \left(\frac{q+1}{2^h} + \frac{2q+1}{2^{h-1}} + \frac{q+\epsilon\ell q(p-1)}{p-1-q} \right) \right).$$

Proof. The proof proceeds in two steps. We first bound the probability that an adversary tampers with the unicorn protocol and we then upper bound $\text{Adv}_{\text{XKEY}_{k, \kappa, P, h, h, p, \ell}}^{\text{key}}(\mathcal{B})$ using techniques from BKR [13, Proof of Theorem 12]. We present the full proof in Section B.5. $\square$

PRACTICAL INSTANTIATION. Communication secured with SES and unicorn is both practical and deployable. A secure unicorn instance (as proposed by LW)

runs every 15 minutes, with a two-minute seed contribution window followed by a 10-minute generation phase. To start a session, both parties contribute to the next seed collection and use the resulting randomness. Contributions are posted in a publicly observable way (e.g., on Mastodon). Multiple staggered *unicorn* instances (e.g., one per minute) or ad-hoc instances are also possible.

As shown by BPR, any efficient nonce-based scheme that is *tidy* [40] produces unique ciphertexts [14], making it suitable for SES. Most standard schemes such as OCB [35] and GCM [37] are tidy. For GCM, if (K, N, A, C) decrypts to some $M \neq \perp$, then $C = \mathcal{E}(K, N, A, M)$: the core ciphertext is computed as XOR of M and a keys stream that is a deterministic function of K, N, A , which will cancel out with the same key stream that has been XORed to C in the decryption yielding M . If K, N, A and the core ciphertext are equal, the tag, computed as a masked keyed hash dependent on the key, the nonce, the associated data and the core ciphertext will be the same as well.

Modern encryption schemes with 128-bit keys can securely process at least 280 TB⁶ of data before rekeying. AES-GCM achieves this by using only 96-bit nonces, which implies birthday security without a quantitative degradation [33]. Over a 1 Gbps connection, transmitting this much data takes $\approx 85 \text{ h} \gg 15 \text{ min}$, well above the *unicorn* refresh interval.

7 Conclusion and future work

In this paper we show that coordinated ASAs and KE attacks render standalone symmetric encryption defenseless, but coupling encryption with verifiable external randomness restores security, thereby charting a concrete path toward cryptography resistant to mass-surveillance.

There are several avenues for further improvements. Like BKR, our model lets the adversary leak information only *before* communication; a more realistic model would allow the APT to leak very small amounts of data between queries. Future work could also relax the decryptability condition [23], allowing stronger subversions [46] or adopt a faster source of verifiable randomness. A natural avenue is to reinterpret our current *unicorn* primitive as a verifiable delay function (VDF) [16]; substituting a faster VDF—or the distributed randomness beacons surveyed by Choi et al. [21]—could supply the same incorruptible randomness with lower latency. Finally, our quantitative bound degrades quickly; tightening it remains an important goal for future work.

Acknowledgments. Damian Vizár’s research was partly supported by the EU H2020 TREDISEC project, funded by the European Commission under grant agreement no. 644412. Simone Colombo’s research was partly supported by UKRI grant EP/X017524/1. We would like to thank the EUROCRYPT and LATINCRYPT reviewers for their constructive comments. We thank Phillip Rogaway, Guy Barwell and Varun Maram for insightful comments and discussion.

⁶ An AES mode of operation with birthday-bounded security can process about 2^{44} blocks of 128 bits each while still having $\approx 2^{-20}$ provable bound on the adversarial advantage. This is about 280 TB.

References

1. Alwen, J., Dodis, Y., Naor, M., Segev, G., Walfish, S., Wichs, D.: Public-key encryption in the bounded-retrieval model. In: *Advances in Cryptology - EURO-CRYPTO 2010, 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, French Riviera, May 30 - June 3, 2010. Proceedings. Lecture Notes in Computer Science, vol. 6110, pp. 113–134. Springer (2010)
2. Alwen, J., Dodis, Y., Wichs, D.: Leakage-resilient public-key cryptography in the bounded-retrieval model. In: *Advances in Cryptology - CRYPTO 2009, 29th Annual International Cryptology Conference*, Santa Barbara, CA, USA, August 16–20, 2009. Proceedings. Lecture Notes in Computer Science, vol. 5677, pp. 36–54. Springer (2009)
3. Alwen, J., Dodis, Y., Wichs, D.: Survey: Leakage resilience and the bounded retrieval model. In: *Information Theoretic Security, 4th International Conference, ICITS 2009, Shizuoka, Japan, December 3–6, 2009. Revised Selected Papers*. Lecture Notes in Computer Science, vol. 5973, pp. 1–18. Springer (2010)
4. Armour, M., Poettering, B.: Algorithm substitution attacks against receivers. *Int. J. Inf. Sec.* **21**(5), 1027–1050 (2022)
5. Ateniese, G., Magri, B., Venturi, D.: Subversion-resilient signature schemes. In: *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, Denver, CO, USA, October 12–6, 2015. pp. 364–375. ACM (2015)
6. Austrin, P., Chung, K., Mahmoody, M., Pass, R., Seth, K.: On the impossibility of cryptography with tamperable randomness. In: *Advances in Cryptology - CRYPTO 2014 - 34th Annual Cryptology Conference*, Santa Barbara, CA, USA, August 17–21, 2014, Proceedings, Part I. Lecture Notes in Computer Science, vol. 8616, pp. 462–479. Springer (2014)
7. Ball, J., Borger, J., Greenwald, G.: Revealed: how US and UK spy agencies defeat internet privacy and security. *The Guardian*, <https://www.theguardian.com/world/2013/sep/05/nsa-gchq-encryption-codes-security> (Sep 2013), last accessed: 10-05-2017
8. Banfi, F., Maurer, U.: Anonymous symmetric-key communication. In: *SCN. Lecture Notes in Computer Science*, vol. 12238, pp. 471–491. Springer (2020)
9. Bellare, M., Desai, A., Jokipii, E., Rogaway, P.: A concrete security treatment of symmetric encryption. In: *38th Annual Symposium on Foundations of Computer Science, FOCS '97, Miami Beach, Florida, USA, October 19–22, 1997*. pp. 394–403. IEEE Computer Society (1997)
10. Bellare, M., Desai, A., Jokipii, E., Rogaway, P.: A concrete security treatment of symmetric encryption. In: *FOCS*. pp. 394–403. IEEE Computer Society (1997)
11. Bellare, M., Hoang, V.T.: Resisting randomness subversion: Fast deterministic and hedged public-key encryption in the standard model. In: *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Sofia, Bulgaria, April 26–30, 2015, Proceedings, Part II. Lecture Notes in Computer Science, vol. 9057, pp. 627–656. Springer (2015)
12. Bellare, M., Jaeger, J., Kane, D.: Mass-surveillance without the state: Strongly undetectable algorithm-substitution attacks. In: *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, Denver, CO, USA, October 12–6, 2015. pp. 1431–1440. ACM (2015)

13. Bellare, M., Kane, D., Rogaway, P.: Big-key symmetric encryption: Resisting key exfiltration. In: *Advances in Cryptology - CRYPTO 2016 - 36th Annual International Cryptology Conference*, Santa Barbara, CA, USA, August 14-18, 2016, Proceedings, Part I. *Lecture Notes in Computer Science*, vol. 9814, pp. 373–402. Springer (2016)
14. Bellare, M., Paterson, K.G., Rogaway, P.: Security of symmetric encryption against mass surveillance. In: *Advances in Cryptology - CRYPTO 2014 - 34th Annual Cryptology Conference*, Santa Barbara, CA, USA, August 17-21, 2014, Proceedings, Part I. *Lecture Notes in Computer Science*, vol. 8616, pp. 1–19. Springer (2014)
15. Berndt, S., Wichelmann, J., Pott, C., Traving, T., Eisenbarth, T.: ASAP: algorithm substitution attacks on cryptographic protocols. In: *AsiaCCS*. pp. 712–726. ACM (2022)
16. Boneh, D., Bonneau, J., Bünz, B., Fisch, B.: Verifiable delay functions. In: *CRYPTO (1)*. *Lecture Notes in Computer Science*, vol. 10991, pp. 757–788. Springer (2018)
17. Camenisch, J., Drijvers, M., Lehmann, A.: Anonymous attestation with subverted tpms. In: *Advances in Cryptology - CRYPTO 2017 - 37th Annual International Cryptology Conference*, Santa Barbara, CA, USA, August 20-24, 2017, Proceedings, Part III. *Lecture Notes in Computer Science*, vol. 10403, pp. 427–461. Springer (2017)
18. Cash, D., Ding, Y.Z., Dodis, Y., Lee, W., Lipton, R.J., Walfish, S.: Intrusion-resilient key exchange in the bounded retrieval model. In: *Theory of Cryptography, 4th Theory of Cryptography Conference, TCC 2007, Amsterdam, The Netherlands, February 21-24, 2007, Proceedings*. *Lecture Notes in Computer Science*, vol. 4392, pp. 479–498. Springer (2007)
19. Champion, J., Waters, B., Wu, D.J.: Adaptively-secure big-key identity-based encryption. In: Jager, T., Pan, J. (eds.) *Public-Key Cryptography – PKC 2025*. pp. 125–157. Springer Nature Switzerland, Cham (2025)
20. Checkoway, S., Niederhagen, R., Everspaugh, A., Green, M., Lange, T., Ristenpart, T., Bernstein, D.J., Maskiewicz, J., Shacham, H., Fredrikson, M.: On the Practical Exploitability of Dual EC in TLS Implementations. In: *USENIX Security* (2014)
21. Choi, K., Manoj, A., Bonneau, J.: Sok: Distributed randomness beacons. In: *SP*. pp. 75–92. IEEE (2023)
22. Crescenzo, G.D., Lipton, R.J., Walfish, S.: Perfectly secure password protocols in the bounded retrieval model. In: *Theory of Cryptography, Third Theory of Cryptography Conference, TCC 2006, New York, NY, USA, March 4-7, 2006, Proceedings*. *Lecture Notes in Computer Science*, vol. 3876, pp. 225–244. Springer (2006)
23. Degabriele, J.P., Farshim, P., Poettering, B.: A more cautious approach to security against mass surveillance. In: *Fast Software Encryption - 22nd International Workshop, FSE 2015, Istanbul, Turkey, March 8-11, 2015, Revised Selected Papers*. *Lecture Notes in Computer Science*, vol. 9054, pp. 579–598. Springer (2015)
24. Dodis, Y., Mironov, I., Stephens-Davidowitz, N.: Message transmission with reverse firewalls - secure communication on corrupted machines. In: *Advances in Cryptology - CRYPTO 2016 - 36th Annual International Cryptology Conference*, Santa Barbara, CA, USA, August 14-18, 2016, Proceedings, Part I. *Lecture Notes in Computer Science*, vol. 9814, pp. 341–372. Springer (2016)
25. Döttling, N., Garg, S., Sekar, S., Wang, M.: IBE with incompressible master secret and small identity secrets. In: *TCC (1)*. *Lecture Notes in Computer Science*, vol. 13747, pp. 588–617. Springer (2022)

26. Dziembowski, S.: Intrusion-resilience via the bounded-storage model. In: TCC. Lecture Notes in Computer Science, vol. 3876, pp. 207–224. Springer (2006)
27. Farrell, S., Badii, F., Schneier, B., Bellare, S.M.: Reflections on ten years past the Snowden revelations. RFC 9446 (2023), <https://www.rfc-editor.org/info/rfc9446>
28. Gellman, B., Nakashima, E.: U.S. spy agencies mounted 231 offensive cyber-operations in 2011, documents show. The Washington Post, https://www.washingtonpost.com/world/national-security/us-spy-agencies-mounted-231-offensive-cyber-operations-in-2011-documents-show/2013/08/30/d090a6ae-119e-11e3-b4cb-fd7ce041d814_story.html (Aug 2013), last accessed: 13-05-2017
29. Green, M.: A case against security nihilism. <https://blog.cryptographyengineering.com/2021/07/20/a-case-against-security-nihilism/> (July 2021), accessed: 24-05-2022
30. Grubbs, P., Lu, J., Ristenpart, T.: Message franking via committing authenticated encryption. In: CRYPTO (3). Lecture Notes in Computer Science, vol. 10403, pp. 66–97. Springer (2017)
31. Hoang, V.T., Reyhanitabar, R., Rogaway, P., Vizár, D.: Online authenticated-encryption and its nonce-reuse misuse-resistance. In: Advances in Cryptology - CRYPTO 2015 - 35th Annual Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2015, Proceedings, Part I. Lecture Notes in Computer Science, vol. 9215, pp. 493–517. Springer (2015)
32. IACR: IACR Statement on Mass Surveillance (“Copenhagen Resolution”). <https://iacr.org/misc/statement-May2014.html> (2014), adopted at the IACR Members meeting, Eurocrypt 2014, Copenhagen, May 14th 2014
33. Iwata, T., Ohashi, K., Minematsu, K.: Breaking and repairing GCM security proofs. In: CRYPTO. Lecture Notes in Computer Science, vol. 7417, pp. 31–49. Springer (2012)
34. Joux, A., Loss, J., Wagner, B.: Kleptographic attacks against implicit rejection. In: Jager, T., Pan, J. (eds.) Public-Key Cryptography – PKC 2025. pp. 214–245. Springer Nature Switzerland, Cham (2025)
35. Krovetz, T., Rogaway, P.: The Software Performance of Authenticated-Encryption Modes. In: Joux, A. (ed.) FSE 2011. LNCS, vol. 6733, pp. 306–327. Springer (2011)
36. Lenstra, A.K., Wesolowski, B.: A random zoo: sloth, unicorn, and trx. IACR Cryptology ePrint Archive **2015**, 366 (2015), <http://eprint.iacr.org/2015/366>
37. McGrew, D.A., Viega, J.: The security and performance of the galois/counter mode (GCM) of operation. In: INDOCRYPT 2004. pp. 343–355 (2004)
38. Mironov, I., Stephens-Davidowitz, N.: Cryptographic reverse firewalls. In: Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part II. Lecture Notes in Computer Science, vol. 9057, pp. 657–686. Springer (2015)
39. Moran, T., Wichs, D.: Incompressible encodings. In: CRYPTO (1). Lecture Notes in Computer Science, vol. 12170, pp. 494–523. Springer (2020)
40. Namprepmpre, C., Rogaway, P., Shrimpton, T.: Reconsidering generic composition. In: Advances in Cryptology - EUROCRYPT 2014 - 33rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Copenhagen, Denmark, May 11-15, 2014. Proceedings. Lecture Notes in Computer Science, vol. 8441, pp. 257–274. Springer (2014)
41. NIST: NIST randomness beacon. <https://beacon.nist.gov>, last accessed 14-09-2017

42. Preneel, B.: Cryptography and information security in the post-snowden era. In: TELERISE@ ICSE. p. 1 (2015)
43. Rabin, M.O.: Transaction protection by beacons. *Journal of Computer and System Sciences* **27**(2), 256–267 (1983)
44. Rogaway, P.: Nonce-based symmetric encryption. In: FSE. *Lecture Notes in Computer Science*, vol. 3017, pp. 348–359. Springer (2004)
45. Rogaway, P.: The moral character of cryptographic work. *IACR Cryptology ePrint Archive* **2015**, 1162 (2015), <http://eprint.iacr.org/2015/1162>
46. Russell, A., Tang, Q., Yung, M., Zhou, H.: Cliptography: Clipping the power of kleptographic attacks. In: *Advances in Cryptology - ASIACRYPT 2016 - 22nd International Conference on the Theory and Application of Cryptology and Information Security*, Hanoi, Vietnam, December 4-8, 2016, Proceedings, Part II. *Lecture Notes in Computer Science*, vol. 10032, pp. 34–64 (2016)
47. Russell, A., Tang, Q., Yung, M., Zhou, H.: Destroying steganography via amalgamation: Kleptographically CPA secure public key encryption. *IACR Cryptology ePrint Archive* **2016**, 530 (2016), <http://eprint.iacr.org/2016/530>
48. Russell, A., Tang, Q., Yung, M., Zhou, H.: Correcting subverted random oracles. In: *CRYPTO (2)*. *Lecture Notes in Computer Science*, vol. 10992, pp. 241–271. Springer (2018)
49. Russell, A., Tang, Q., Yung, M., Zhou, H.: Correcting subverted random oracles. In: *CRYPTO (2)*. *Lecture Notes in Computer Science*, vol. 10992, pp. 241–271. Springer (2018)
50. Scott, A., Andersen, S.: Engineering a backdoored bitcoin wallet. In: *WOOT @ USENIX Security Symposium*. pp. 89–100. USENIX Association (2024)
51. Scott-Railton, J., Campo, E., Marczak, B., Razzak, B.A., Anstis, S., Böcü, G., Solimano, S., Deibert, R.: CatalanGate: Extensive Mercenary Spyware Operation against Catalans Using Pegasus and Candiru. <https://citizenlab.ca/2022/04/catalangate-extensive-mercenary-spyware-operation-against-catalans-using-pegasus-candiru/> (2022), accessed: 22-05-2022
52. Vaudenay, S.: Clever arbiters versus malicious adversaries - on the gap between known-input security and chosen-input security. In: *The New Codebreakers - Essays Dedicated to David Kahn on the Occasion of His 85th Birthday*. *Lecture Notes in Computer Science*, vol. 9100, pp. 497–517. Springer (2016)
53. Young, A.L., Yung, M.: The dark side of "black-box" cryptography, or: Should we trust capstone? In: *Advances in Cryptology - CRYPTO '96, 16th Annual International Cryptology Conference*, Santa Barbara, California, USA, August 18-22, 1996, Proceedings. *Lecture Notes in Computer Science*, vol. 1109, pp. 89–103. Springer (1996)
54. Young, A.L., Yung, M.: Kleptography: Using cryptography against cryptography. In: *Advances in Cryptology - EUROCRYPT '97, International Conference on the Theory and Application of Cryptographic Techniques*, Konstanz, Germany, May 11-15, 1997, Proceeding. *Lecture Notes in Computer Science*, vol. 1233, pp. 62–74. Springer (1997)

A Deferred Definitions and Lemmas

Definition 13 (Unique ciphertexts [14]). Let $\Pi = (\mathcal{K}, \mathcal{E}, \mathcal{D})$ be a symmetric encryption scheme. For any possible state $\tau \in \mathcal{S}$ of $\mathcal{D}$ with respect to key K , any message $M \in \mathcal{M}$ and any associated data $A \in \mathcal{A}$, let $\mathcal{C}_\Pi(K, M, A, \tau)$ be the set of all ciphertexts C such that $(M, \tau') \leftarrow \mathcal{D}(K, C, A, \tau)$ for some $\tau' \in \mathcal{S}$. We say that Π has unique ciphertexts if $|\mathcal{C}_\Pi(K, M, A, \tau)| \leq 1$ for all K, M, A, τ .

Lemma 6 ([13]). Let k, κ, P, r be integers with k a power of two. Let $\mathcal{A}$ be an adversary that leaks at most l bits and makes at most q DERIVE queries. Let the number of RO queries made by both the leakage function and $\mathcal{B}$ before $\mathcal{B}$ receives the leakage be at most q_1 and the number of RO queries made by $\mathcal{B}$ after it receives the leakage be at most q_2 . Then for the scheme XKEY (Fig. 4) we have

$$\mathbf{Adv}_{\text{XKEY}_{k, \kappa, P, r}}^{\text{key}}(\mathcal{B}) \lesssim q_2 \cdot q \cdot 2^{-P \cdot w(l/k)} + \frac{q \cdot (2q_1 + q - 1)}{2^{r+1}},$$

with $w(\lambda) = -\log_2(1 - H_2^{-1}(1 - \lambda))$, where H_2 is the binary entropy function and $H_2^{-1}(1 - \lambda)$ is the smaller of the two possible inverses of $1 - \lambda$ under H_2 .

B Deferred Proofs

B.1 Proof of Proposition 2

Proof (Proposition 2). Let $\mathcal{B}$ be an adversary that plays the LIND-ROR game of Fig. 6 and let its advantage be $\mathbf{Adv}_\Pi^{\text{lind-ror}}(\mathcal{B}) = 2 \Pr[\text{LIND-ROR}_\Pi^{\mathcal{B}} \Rightarrow 1] - 1$.

Given $\mathcal{B}$, we can build an adversary $\mathcal{B}'$ that plays the LIND\$ game as follows. The LIND\$ adversary $\mathcal{B}'$ samples a random bit $b^* \leftarrow_{\$} \{0, 1\}$ and when $b^* = 1$ it answers to $\mathcal{B}$'s query using its own oracle ENC on the message that $\mathcal{B}$ submits, when $b^* = 0$ it samples a random message M' and answers to $\mathcal{B}$ by feeding M' to the encryption oracle. When $\mathcal{B}$ returns its bit $\tilde{b}$, the LIND\$ adversary $\mathcal{B}'$ returns $(b^* = \tilde{b})$, i.e., returns 1 if $b^* = \tilde{b}$ and 0 otherwise.

We can compute the advantage of $\mathcal{B}'$ as follows:

$$\begin{aligned} \mathbf{Adv}_\Pi^{\text{lind\$}}(\mathcal{B}') &= 2 \Pr[\text{LIND\$}_\Pi^{\mathcal{B}'} \Rightarrow 1] - 1 = \left| 2 \Pr[\text{LIND\$}_\Pi^{\mathcal{B}'} \Rightarrow 1] - 1 \right| \\ &= |2 \cdot (\Pr[b' = 1|b = 1] \cdot 1/2 + \Pr[b' = 0|b = 0] \cdot 1/2) - 1| \\ &= |\Pr[b' = 1|b = 1] + \Pr[b' = 0|b = 0] - 1| \\ &= |\Pr[b' = 1|b = 1] - \Pr[b' = 1|b = 0]| \\ &= |\Pr[b^* = \tilde{b}|b = 1] - \Pr[b^* = \tilde{b}|b = 0]| \\ &= |1/2 - \Pr[b^* = \tilde{b}|b = 0]|, \end{aligned}$$

where the last equality holds since only random guessing is available to the LIND-ROR adversary as the LIND\$ adversary samples a random message and returns the output of the ENC oracle on the independent random message. Observe that

when $b = 0$ the LIND\$ adversary $\mathcal{B}'$ perfectly simulates the ENC oracle of the LIND-ROR adversary, hence:

$$\begin{aligned}
\mathbf{Adv}_{\Pi}^{\text{lind\$}}(\mathcal{B}') &= |1/2 - \Pr[b^* = \tilde{b} | b = 0]| \\
&= |1/2 - \Pr[\tilde{b} = 1 | b^* = 1] \cdot 1/2 - \Pr[\tilde{b} = 0 | b^* = 0] \cdot 1/2| \\
&= |1/2 \cdot (1 - \Pr[\tilde{b} = 1 | b^* = 1]) - \Pr[\tilde{b} = 0 | b^* = 0]| \\
&= |1/2 \cdot (\Pr[\tilde{b} = 1 | b^* = 1] + \Pr[\tilde{b} = 0 | b^* = 0] - 1)| \\
&= 1/2 \cdot \mathbf{Adv}_{\Pi}^{\text{lind-ror}}(\mathcal{B}).
\end{aligned}$$

Since the LIND-ROR adversary $\mathcal{B}$ is an arbitrary adversary, we have that $\mathbf{Adv}_{\Pi}^{\text{lind-ror}}(\mathcal{B}) \leq 2 \cdot \mathbf{Adv}_{\Pi}^{\text{lind\$}}(\mathcal{B}')$

The proof technique from Bellare et al. [10, Theorem 1] (similar to the one we just used to prove the relation between LIND-ROR security and LIND\$ security) applies to show that $\mathbf{Adv}_{\Pi}^{\text{lind}}(\mathcal{B}) \leq 2 \cdot \mathbf{Adv}_{\Pi}^{\text{lind-ror}}(\mathcal{B})$. Therefore we have $\mathbf{Adv}_{\Pi}^{\text{lind}}(\mathcal{B}) \leq 2 \cdot \mathbf{Adv}_{\Pi}^{\text{lind-ror}}(\mathcal{B}) \leq 4 \cdot \mathbf{Adv}_{\Pi}^{\text{lind\$}}(\mathcal{B})$. $\square$

B.2 Proof of Lemma 1

Game $\overline{\text{ASA}}_{\Pi, \tilde{\Pi}} \left(\mathbf{K}, \iota, q, R_1^q, \vec{q}, \left(M_{i,1}^{i, \vec{q}[i]} \right)_{i \in [1..q]}, \left(A_{i,1}^{i, \vec{q}[i]} \right)_{i \in [1..q]} \right)$	
1 :	$\sigma_{0,0}, \tilde{\sigma}_{0,0}, \tau_{0,0}, \tilde{\tau}_{0,0} \leftarrow \perp; \vec{q}[0] \leftarrow 0$
2 :	for $i \leftarrow 1, \dots, q$ do
3 :	$\sigma_{i,0} \leftarrow \mathcal{E}.\text{Init}^{\text{RO}}(\mathbf{K}, R_i, \sigma_{i-1}, \vec{q}[i-1])$
4 :	$\tilde{\sigma}_{i,0} \leftarrow \tilde{\mathcal{E}}.\text{Init}^{\text{RO}}(\tilde{\mathbf{K}}, \mathbf{K}, R_i, \tilde{\sigma}_{i-1}, \vec{q}[i-1], \iota)$
5 :	$\tau_{i,0} \leftarrow \mathcal{D}.\text{Init}^{\text{RO}}(\mathbf{K}, R_i, \tau_{i-1}, \vec{q}[i-1])$
6 :	$\tilde{\tau}_{i,0} \leftarrow \mathcal{D}.\text{Init}^{\text{RO}}(\mathbf{K}, R_i, \tilde{\tau}_{i-1}, \vec{q}[i-1])$
7 :	for $j \leftarrow 1, \dots, \vec{q}[i]$ do
8 :	$(C_{i,j}, \sigma_{i,j}) \leftarrow \mathcal{E}.\text{Enc}(\mathbf{K}, M_{i,j}, A_{i,j}, \sigma_{i,j-1})$
9 :	$(\tilde{C}_{i,j}, \tilde{\sigma}_{i,j}) \leftarrow \tilde{\mathcal{E}}.\text{Enc}(\tilde{\mathbf{K}}, \mathbf{K}, M_{i,j}, A_{i,j}, \tilde{\sigma}_{i,j-1}, \iota)$
10 :	$(M'_{i,j}, \tau_{i,j}) \leftarrow \mathcal{D}.\text{Dec}(\mathbf{K}, C_{i,j}, A_{i,j}, \tau_{i,j-1})$
11 :	$(\tilde{M}_{i,j}, \tilde{\tau}_{i,j}) \leftarrow \mathcal{D}.\text{Dec}(\mathbf{K}, \tilde{C}_{i,j}, A_{i,j}, \tilde{\tau}_{i,j-1})$
12 :	return $(\exists i, j: C_{i,j} \neq \tilde{C}_{i,j})$

Fig. 19: The game formalizing ineffectiveness of ASAs (excludes the boxed lines).

Proof (Lemma 1). We show that a subversion of $\text{SES}[\Pi, k, P]$ that meets the decryptability condition must always output the same ciphertexts as the original

scheme $\text{SES}[\Pi, k, P]$ would, in any possible sequence of sessions. The proof is a BKSS-variant of BPR's argument about SURV security of schemes with unique ciphertexts. Formally, we claim that

$$\begin{aligned} & \forall \tilde{K} \in \tilde{\mathcal{K}}, \forall \mathbf{K} \in \mathcal{K}, \forall \iota \in \mathcal{I}, \forall q \in \mathbb{N}, \forall (R_1, \dots, R_q) \in \mathcal{R}^q, \forall \vec{q} \in \mathbb{N}^q, \\ & \forall ((M_{1,1}, \dots, M_{1,\vec{q}[1]}), \dots, (M_{1,q}, \dots, M_{1,\vec{q}[q]})) \in \mathcal{M}^{\vec{q}[1]} \times \dots \times \mathcal{M}^{\vec{q}[q]}, \\ & \forall ((A_{1,1}, \dots, A_{1,\vec{q}[1]}), \dots, (A_{1,q}, \dots, A_{1,\vec{q}[q]})) \in \mathcal{A}^{\vec{q}[1]} \times \dots \times \mathcal{A}^{\vec{q}[q]} \end{aligned}$$

we have

$$\Pr[\overline{\text{ASA}}_{\Pi, \tilde{\Pi}}(\mathbf{K}, \iota, q, R_1^q, \vec{q}, (M_{i,1}^{\vec{q}[i]})_{i \in [1..q]}, (A_{i,1}^{\vec{q}[i]})_{i \in [1..q]}) \Rightarrow 1] = 0,$$

with the game $\overline{\text{ASA}}$ defined in Fig. 19. To prove this claim, we extend $\overline{\text{ASA}}$ with the boxed statements in Fig. 19. We prove that $C_{i,j} = \tilde{C}_{i,j}$ and $\tau_{i,j} = \tilde{\tau}_{i,j}$ for all $\{(i,j)\}_{i=1,j=1}^{q,\vec{q}[q]}$ by induction. The induction assumption is that for all $(i',j') < (i,j)$ we have $C_{i',j'} = \tilde{C}_{i',j'}$ and $\tau_{i',j'} = \tilde{\tau}_{i',j'}$, which is trivially true for $(i,j) = (1,1)$. We have two cases of induction step to consider.

- case $1 < j \leq \vec{q}[i]$:** By induction assumption, we have $\tau_{i,j-1} = \tilde{\tau}_{i,j-1} = K \| s$ for some $K \in \mathcal{K}$ and $s \in \mathcal{S}_\Pi$ ($\mathcal{S}_\Pi$ is the state space of Π). If the subversion $\tilde{\Pi}$ is to meet the decryptability condition, we must have $\tilde{M}_{i,1} = M_{i,1}$. But due to correctness of Π , we must have $M'_{i,j} = M_{i,j}$. Because we decrypt $\tilde{M}_{i,j} = \mathcal{D}.\text{Dec}(\mathbf{K}, \tilde{C}_{i,j}, A_{i,j}, \tilde{\tau}_{i,j-1}) = \mathcal{D}(K, \tilde{C}_{i,j}, A_{i,j}, s)$, we must have $\tilde{C}_{i,j} \in \mathcal{C}_\Pi(K, M_{i,j}, A_{i,j}, s) = \mathcal{C}_\Pi(K, M'_{i,j}, A_{i,j}, s) = \{C_{i,j}\}$, as Π has unique ciphertexts. Thus $\tilde{C}_{i,j} = C_{i,j}$, and all the inputs to $\mathcal{D}.\text{Dec}$ that produce $M'_{i,j}, \tau_{i,j}$ are equal to those that produce $\tilde{M}_{i,j}, \tilde{\tau}_{i,j}$, so necessarily $\tau_{i,j} = \tilde{\tau}_{i,j}$.
- case $j = 1$:** By induction assumption, we have $\tau_{i-1,\vec{q}[i-1]} = \tilde{\tau}_{i-1,\vec{q}[i-1]}$. Running the algorithm $\mathcal{D}.\text{Init}^{\text{RO}}$ twice with the same inputs and the same RO always yields the same result, and thus $\tau_{i,0} = \tilde{\tau}_{i,0} = K_i \| \varepsilon$. The rest of the argument for this case is identical with the previous case.

As the subversion $\tilde{\Pi}$ and the original scheme Π produce the same ciphertexts for any sequence of sessions, replacing $\tilde{\Pi}$ with Π in RESIST will yield a perfectly indistinguishable game and the desired result follows. $\square$

B.3 Proof of Lemma 2

Proof (Lemma 2). Without loss of generality assume that $\mathcal{B}$ uses a distinct user identifier in each LEAK query. Let $\iota_1, \dots, \iota_{q_k}$ be these q_k identifiers (submitted by $\mathcal{B}$ in this order). We define $q_k + 1$ hybrids $\text{RST1}.j$ for $j = 0, \dots, q_k$. In the game $\text{RST1}.j$, the queries $\text{INIT}(\hat{\tau}, \iota_u)$ are processed as in RST2 (the derived key K is uniform) for $1 \leq u \leq j$, while for $j < u \leq q_k$ they are processed as in RST1 (K is derived using XKEY). We have $\text{RST1} = \text{RST1}.0$ and $\text{RST2} = \text{RST1}.q_k$.

Omitting the game parameters for compactness, we have that

$$\begin{aligned} \sum_{\beta=0}^1 (-1)^{1-\beta} \left(\Pr[\text{RST1}_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow \beta] - \Pr[\text{RST2}_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow \beta] \right) = \\ \sum_{j=1}^{q_k} \sum_{\beta=0}^1 (-1)^{1-\beta} \left(\Pr[\text{RST1}.(j-1)_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow \beta] - \Pr[\text{RST1}.j_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow \beta] \right). \end{aligned}$$

We show that

$$\begin{aligned} \left| \sum_{\beta=0}^1 (-1)^{1-\beta} \left(\Pr[\text{RST1}.(j-1)_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow \beta] - \Pr[\text{RST1}.j_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow \beta] \right) \right| \leq \\ 2 \cdot \left| \Pr[\text{UKEY}_{\text{XKEY}_{k, \kappa, P}}^{\mathcal{A}} \Rightarrow 1] - \Pr[\text{UKEY}_{\text{XKEY}_{k, \kappa, P}}^{\mathcal{A}} \Rightarrow 0] \right| \end{aligned}$$

and the lemma follows by triangular inequality.

The adversary $\mathcal{A}$ simulates either $\text{RST1}.(j-1)$ or $\text{RST1}.j$ for $\mathcal{B}$, depending on the challenge bit in UKEY (Fig. 15). It is constructed as follows: $\mathcal{A}$ runs $\mathcal{B}$ with its own oracles in $\mathcal{O}$ to obtain the leakage function Lk from $\mathcal{B}$, and submits it as its own. Adversary $\mathcal{A}$ then samples $\bar{K}$ and the bit $\bar{b}$ for $\mathcal{B}$ to guess and simulates the rest of the game for $\mathcal{B}$.

On a query $\text{INIT}(\hat{\tau}, \iota_u)$ made by $\mathcal{B}$, $\mathcal{A}$ proceeds as in RST2 for $1 \leq u < j$. For $j < u \leq q_k$ $\mathcal{A}$ proceeds as in RST1 . If $\mathcal{B}$'s queries result in an abort, $\mathcal{A}$ gives up. Whenever $\mathcal{B}$ makes a query $\text{INIT}(\hat{\tau}, \iota_j)$, $\mathcal{A}$ makes a $\text{DERIVE}(\hat{\tau})$ query to obtain $\hat{\tau}, R, K$, sets $\sigma_{\iota_j} \leftarrow K \parallel \varepsilon$ and returns $\hat{\tau}, R$ to $\mathcal{B}$.

The LEAK and ENC oracles are simulated as in $\text{RST1}/\text{RST2}$, except for the query $\text{LEAK}(\iota_j)$, which $\mathcal{A}$ replies with the leakage it received at the beginning of the UKEY game. When $\mathcal{B}$ outputs its final bit b' , $\mathcal{A}$ computes the output of the game $b' = 1 \oplus (\bar{b} \oplus \bar{b}')$ and outputs b' .

If the challenge bit in UKEY $b = 1$, $\mathcal{A}$ perfectly simulates $\text{RST1}.(j-1)$. If $b = 0$, $\mathcal{A}$ perfectly simulates $\text{RST1}.j$. With a slight abuse of notation, we have

$$\begin{aligned} \Pr[\text{UKEY}_{\text{XKEY}}^{\mathcal{A}} \Rightarrow 1] &= \frac{1}{2} \left(\Pr[\text{RST1}.(j-1)_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow 1] + \Pr[\text{RST1}.(j)_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow 0] \right), \\ \Pr[\text{UKEY}_{\text{XKEY}}^{\mathcal{A}} \Rightarrow 0] &= \frac{1}{2} \left(\Pr[\text{RST1}.(j-1)_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow 0] + \Pr[\text{RST1}.(j)_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow 1] \right). \end{aligned}$$

By substituting the terms in $\Pr[\text{UKEY}_{\text{XKEY}}^{\mathcal{A}} \Rightarrow 1] - \Pr[\text{UKEY}_{\text{XKEY}}^{\mathcal{A}} \Rightarrow 0]$ we obtain an equality, which implies the desired result. $\square$

B.4 Proof of Lemma 3

Proof (Lemma 3). We define $q_I + 1$ hybrids $\text{RST2}.j$ for $j = 0, \dots, q_I$. A hybrid game $\text{RST2}.j$ is exactly the same as RST2 , except that the ENC queries belonging to the first j sessions are answered using M_1 , while the ENC queries belonging to the remaining $q_I - j$ sessions are answered using M_0 (regardless of the bit b). When $\mathcal{B}$ issues its output bit b' , $\text{RST2}.j$ simply outputs b' as well. Omitting the game parameters for compactness, we have $\Pr[\text{RST2}^{\mathcal{B}} \Rightarrow 1] =$

$$\Pr[\text{RST2}.q_I^{\mathcal{B}} \Rightarrow 1] + \Pr[\text{RST2}.0^{\mathcal{B}} \Rightarrow 0] \text{ and } \Pr[\text{RST2}^{\mathcal{B}} \Rightarrow 0] = \Pr[\text{RST2}.q_I^{\mathcal{B}} \Rightarrow 0] + \Pr[\text{RST2}.0^{\mathcal{B}} \Rightarrow 1].$$

By further letting $D(j) = \Pr[\text{RST2}.j^{\mathcal{B}} \Rightarrow 1] - \Pr[\text{RST2}.j^{\mathcal{B}} \Rightarrow 0]$ we have that

$$\Pr[\text{RST2}_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow 1] - \Pr[\text{RST2}_{\Pi, \tilde{\Pi}}^{\mathcal{B}} \Rightarrow 0] = \sum_{j=1}^{q_I} D(j) - D(j-1).$$

We now construct an adversary $\mathcal{A}$ that uses Big Brother $\mathcal{B}$ as a subroutine and has $\mathbf{Adv}_{\Pi}^{\text{ind\$}}(\mathcal{A}) \geq D(j) - D(j-1)$. The lemma will follow by triangular inequality.

$\mathcal{A}$ samples $\tilde{K}$, runs $\mathcal{B}$, simulates RO by lazy sampling and simulates the LEAK oracle following the code of RST2. $\mathcal{A}$ simulates the first $j-1$ INIT queries to compute the session keys, and uses these to simulate the ENC queries for the corresponding sessions, always computing the ciphertexts by using the $\mathcal{E}.\text{Enc}$ algorithm of Π with the session key that $\mathcal{A}$ computes by simulating INIT. When the j^{th} INIT($\iota_j, \hat{\tau}$) query is made, $\mathcal{A}$ only computes the values $R, \hat{\tau}$ to be returned to $\mathcal{B}$, and answers all the ENC(M, A, ι_j) queries in the j^{th} session with its own ENC oracle. $\mathcal{A}$ ignores the remaining $q_I - j$ INIT queries, since $\mathcal{A}$ does not need the session keys, as it simulates the corresponding ENC queries by always returning a random bit string $C_1 \leftarrow \{0, 1\}^c$, where c is the length of the ciphertexts that $\mathcal{E}.\text{Enc}$ outputs. When $\mathcal{B}$ outputs its final bit b' , $\mathcal{A}$ outputs b' as well. If the simulation aborts, $\mathcal{A}$ outputs a random bit.

We have $\mathbf{Adv}_{\Pi}^{\text{ind\$}}(\mathcal{A}) = \Pr[\text{IND\$-CPA}_{\Pi}^{\mathcal{A}} \Rightarrow 1] - \Pr[\text{IND\$-CPA}_{\Pi}^{\mathcal{A}} \Rightarrow 0]$. Let A denote the event that the simulation run by $\mathcal{A}$ aborts. Then $\Pr[\text{IND\$-CPA}_{\Pi}^{\mathcal{A}} \Rightarrow 1 \wedge A] - \Pr[\text{IND\$-CPA}_{\Pi}^{\mathcal{A}} \Rightarrow 0 \wedge A] = 0$, as $\mathcal{A}$ returns a random bit whenever an abort occurs. Thus we have $\mathbf{Adv}_{\Pi}^{\text{ind\$}}(\mathcal{A}) = \Pr[\text{IND\$-CPA}_{\Pi}^{\mathcal{A}} \Rightarrow 1 \wedge \neg A] - \Pr[\text{IND\$-CPA}_{\Pi}^{\mathcal{A}} \Rightarrow 0 \wedge \neg A]$.

If the bit $b = 1$ in IND\\$-CPA game, then $\mathcal{A}$ simulates RST2. j for $\mathcal{B}$ and if $b = 0$ it simulates RST2. $(j-1)$. We have

$$\begin{aligned} \Pr[\text{IND\$-CPA}_{\Pi}^{\mathcal{A}} \Rightarrow 1 \wedge \neg A] &= \frac{1}{2} (\Pr[b' = 1 \wedge \neg A | b = 1] + \Pr[b' = 0 \wedge \neg A | b = 0]) \\ &= \frac{1}{2} (\Pr[\text{RST2}.j^{\mathcal{B}} \Rightarrow 1] + \Pr[\text{RST2}.(j-1)^{\mathcal{B}} \Rightarrow 0]) \end{aligned}$$

and

$$\begin{aligned} \Pr[\text{IND\$-CPA}_{\Pi}^{\mathcal{A}} \Rightarrow 0 \wedge \neg A] &= \frac{1}{2} (\Pr[b' = 0 \wedge \neg A | b = 1] + \Pr[b' = 1 \wedge \neg A | b = 0]) \\ &= \frac{1}{2} (\Pr[\text{RST2}.j^{\mathcal{B}} \Rightarrow 0] + \Pr[\text{RST2}.(j-1)^{\mathcal{B}} \Rightarrow 1]). \end{aligned}$$

By applying these equalities to the expression $\mathbf{Adv}_{\Pi}^{\text{ind\$}}(\mathcal{A}) = \Pr[\text{IND\$-CPA}_{\Pi}^{\mathcal{A}} \Rightarrow 1 \wedge \neg A] - \Pr[\text{IND\$-CPA}_{\Pi}^{\mathcal{A}} \Rightarrow 0 \wedge \neg A]$, we obtain $\mathbf{Adv}_{\Pi}^{\text{ind\$}}(\mathcal{A}) = \frac{1}{2}(D(j) - D(j-1))$ which concludes the proof. $\square$

B.5 Proof of Lemma 5

Proof (Lemma 5). Let R_i be the random seed returned by the procedure GENERATE in $\mathcal{A}$'s i^{th} query to the DERIVE oracle. $\text{RST1}_{\Pi, \tilde{\Pi}, h, p, \ell}^{\mathcal{B}}$. We first show that for any $i = 1, \dots, q_I$ and for any $y \in \mathbb{Z}_p^*$

$$\mathbf{P}_R^{i,y} = \Pr[\text{UKEY}_{\text{XKEY}_{k, \kappa, P, h, h, p, \ell}}^{\mathcal{A}} : R_i = y] \leq \frac{q+1}{2^h} + \frac{2q+1}{2^{h-1}} + \frac{q + \epsilon \ell q(p-1)}{p-1-q}$$

by constructing a unicorn-adversary $\mathcal{C}$ that is playing the game $\text{INCORR}_{\mathfrak{P}^{h,p,\ell,\delta,t}}^{\mathcal{C}}$ and uses $\mathcal{A}$ as a subroutine, with the target function t being the indicator function $t(x') = 1$ if and only if $y = x'$.

$\mathcal{C}$ simulates the game $\text{UKEY}_{\text{XKEY}_{k, \kappa, P, h, h, p, \ell}}^{\mathcal{A}}$ for $\mathcal{A}$, using its own oracles RO' , θ , θ^{-1} and implementing RO by lazy sampling, until $\mathcal{A}$ either aborts or issues its i^{th} $\text{DERIVE}(\hat{\tau})$ query. When the $\text{GENERATE}(\hat{\tau})$ procedure in this query starts, $\mathcal{C}$ saves all its variables and $\hat{\tau}$ encoded in its own output-state. Then $\mathcal{C}$ receives the honest seed x in the time-critical phase of the INCORR game, and runs $\hat{\tau}, \bar{s}, c \xleftarrow{T(\mathcal{A}) \leq \delta} \mathcal{A}^{\mathcal{O}}(x, \hat{\tau})$, simply forwards $\bar{s}, c$ and saves all its own variables and $\mathcal{A}$'s updated state $\hat{\tau}$ in its own output-state. $\mathcal{C}$ then runs $\mathcal{A}^{\mathcal{O}}(\hat{\tau})$ again to obtain s_0, R, w , which $\mathcal{A}$ forwards as its own outputs.

We have that $\mathbf{P}_R(i, y) = \Pr[\text{INCORR}_{\mathfrak{P}^{h,p,\ell,\delta,t}}^{\mathcal{C}} \Rightarrow 1]$, because $t(R_i) = 1$ can only occur if UKEY does not abort before R_i is computed and if $R_i \neq \perp$. The desired bound on $\mathbf{P}_R(i, y)$ follows from Lemma 4 and Corollary 1.

The rest of the proof is almost identical with the proof of the upper bound on $\text{Adv}_{\text{XKEY}_{k, \kappa, p, r}}^{\text{key}}(\mathcal{B})$ by BKR [13, Proof of Theorem 12]; the only difference here is that the random seed (selector) R_i is no longer sampled uniformly, because the adversary can tamper with its generation. However, having established $\mathbf{P}_R^{i,y} \leq \frac{q+1}{2^h} + \frac{2q+1}{2^{h-1}} + \frac{q + \epsilon \ell q(p-1)}{p-1-q}$, we simply the uniform probability $1/2^h$ in the original bound by the upper bound on $\mathbf{P}_R^{i,y}$ and the lemma follows. $\square$

C Unicorn: Algorithms and Proof

This section describes the three algorithms used in the unicorn protocol in more detail, and contains the deferred proof of Lemma 4.

C.1 Algorithms used in the unicorn protocol

Internally, $\mathfrak{P}.\text{SH}^{H,p,\theta}$ uses a special permutation $\rho : \mathbb{Z}_p^* \rightarrow \mathbb{Z}_p^*$ based on computing square roots modulo p , defined as

$$\rho(x) = \begin{cases} \sqrt[p]{x} \bmod p & \text{if } x \in \text{QR}_p \\ \sqrt[p]{-x} \bmod p & \text{otherwise} \end{cases} \quad \rho^{-1}(y) = \begin{cases} y^2 \bmod p & \text{if } y \text{ is even} \\ -y^2 \bmod p & \text{if } y \text{ is odd} \end{cases}$$

where for any $x \in \text{QR}_p$, $\sqrt[p]{x}$ is the unique $y \in \mathbb{Z}_p^*$ such that $y^2 \equiv x \pmod{p}$ and y is even, while $\sqrt[p]{-x}$ is the unique $y \in \mathbb{Z}_p^*$ such that $y^2 \equiv x \pmod{p}$ and y is odd. We further let $\text{int} : \{0, 1\}^h$ denote the canonical encoding of h -bit strings

as integers and $\text{hex} : \mathbb{N} \rightarrow \{0, 1\}^*$ denote the canonical injective encoding that represents a positive integer in binary without leading zeros. The algorithms used in the **unicorn** protocol are defined in Fig. 20.

Algorithm $\mathfrak{P}.\text{SH}^{H,p,\ell,\theta}(s)$	Algorithm $\mathfrak{P}.\text{ver}^{H,p,\ell,\theta}(s, c, g, w)$
$u \leftarrow H(s)$	$u \leftarrow H(s)$
$w_0 \leftarrow \text{int}(u)$	if $(c \neq H(u))$ then return 0
for $i \leftarrow 1, \dots, \ell$ do	if $(g \neq H(\text{str}(w)))$ then return 0
$w_i \leftarrow \rho(\theta(w_{i-1}))$	$w_\ell \leftarrow w$
$g \leftarrow H(\text{str}(w_\ell))$	for $i \leftarrow \ell, \dots, 1$ do
return g, w_ℓ	$w_i \leftarrow \theta^{-1}(\rho^{-1}(w_{i+1}))$
Algorithm $\mathfrak{P}.\text{commit}^H(s)$	if $(w_1 \neq \text{int}(u))$ then return 0
return $H(H(s))$	return 1

Fig. 20: The algorithms $\mathfrak{P}.\text{SH}^{H,p,\ell,\theta}$, $\mathfrak{P}.\text{commit}^H(s)$ and $\mathfrak{P}.\text{ver}^{H,p,\ell,\theta}$, parametrized by a conventional hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^h$, a prime number p with $p \geq 2^h$ and $p \equiv 3 \pmod{4}$, a positive integer ℓ , and a permutation $\theta : \mathbb{Z}_p^* \rightarrow \mathbb{Z}_p^*$.

C.2 Proof of Lemma 4

Proof (Lemma 4). Let $\mathcal{Q}$ denote the set of all RO-queries made by $\mathcal{A}$ before it outputs $\sigma, \bar{s}, c$. Let $\mathbf{E}$ be the event that $\bar{s} \| s_0 \in \mathcal{Q}$. By the law of total probability we have $\Pr[\text{INCORR}_{\mathfrak{P}^{p,\ell,\delta,t}}^{\mathcal{B}} \Rightarrow 1] \leq \Pr[\text{INCORR}_{\mathfrak{P}^{p,\ell,\delta,t}}^{\mathcal{B}} \Rightarrow 1 | \mathbf{E}] + \Pr[\text{INCORR}_{\mathfrak{P}^{p,\ell,\delta,t}}^{\mathcal{B}} \Rightarrow 1 | \neg \mathbf{E}]$.

In order for $\text{INCORR}_{\mathfrak{P}^{p,\ell,\delta,t}}^{\mathcal{B}} \Rightarrow 1 | \neg \mathbf{E}$ to occur, $\mathcal{A}$ has to find a value s_0 for which $\mathfrak{P}.\text{commit}^{\text{RO}}(\bar{s} \| s_0) = \text{RO}(\text{RO}(\bar{s} \| s_0)) = c$ for $\bar{s}$ and c which are already fixed. We have that $\Pr[\text{INCORR}_{\mathfrak{P}^{p,\ell,\delta,t}}^{\mathcal{B}} \Rightarrow 1 | \neg \mathbf{E}] \leq \Pr[\text{RO}(\text{RO}(\bar{s} \| s_0)) = c]$. Since $\bar{s} \| s_0$ has not been queried to RO by that moment, $\mathcal{A}$ could try values s_0^j for $j = 1, \dots, r$ with $r \leq q$ to find such s_0 . For any s_0^j , we have

$$\begin{aligned} \Pr[\text{RO}(\text{RO}(\bar{s} \| s_0^j)) = c] &= \sum_{u \in \{0,1\}^h} \Pr[\text{RO}(u) = c] \Pr[\text{RO}(\bar{s} \| s_0^j) = u] \\ &\leq q \cdot 1 \cdot \frac{1}{2^h} + (2^h - q) \cdot \frac{1}{2^h} \cdot \frac{1}{2^h} \leq \frac{q+1}{2^h}. \end{aligned}$$

because $\mathcal{A}$ may have previously discovered at most q inputs $u \in \{0, 1\}^h$ with $\text{RO}(u) = c$.

Otherwise, if $\mathbf{E}$ occurs, we have $\Pr[\text{INCORR}_{\mathfrak{P}^{p,\ell,\delta,t}}^{\mathcal{B}} \Rightarrow 1 | \mathbf{E}] \leq \frac{2q + |t^{-1}(1)|}{2^{h/2}} + \frac{q + \epsilon \ell q(p-1)}{p-1-q}$, where the bound follows from the analysis done by LW [36, Proposition 1].

This is because if s_0 is determined within the δ -time limit, so is $u = \text{RO}(\bar{s} \| s_0)$. Since all the steps that map u to w in the algorithm SH are invertible, g, w must be the output of $\mathfrak{P}.\text{SH}^{p,\ell}(\bar{s} \| s_0)$ (otherwise, the verification would fail). This is the scenario investigated by LW: what is the probability of $t(g) = 1$ if it is truly computed from an adversarial s . $\square$